

DOPart: Competitive-Ratio Optimal Online Offloading of Sequentially Dependent Tasks

Shiva Saxena and Ben Liang
Department of Electrical and Computer Engineering
University of Toronto, Canada

Abstract—This work is motivated by the need for real-time partitioning of a deep neural network inference job and offloading part of it from a resource-constrained device to a resource-rich server. Since both the processing delay of different job stages and the communication delay of offloading usually are unknown ahead of time, this problem is naturally modelled as online offloading of sequentially dependent computational tasks from a local processor to a remote processor. We aim to minimize the total delay of all tasks by utilizing local and remote computational resources, while considering the impact of the offloading delay. We propose Delay-optimal Online Partitioning (DOPart), a lightweight online algorithm that uses an adaptive thresholding strategy to solve the offloading problem. We derive the competitive ratio for DOPart and show that it is optimal in the sense that no other deterministic online algorithm can achieve a lower competitive ratio. Furthermore, it provides bounded performance guarantees even under imprecise parameter estimation. Through experimenting with AlexNet, VGG-19, and ResNet-34, we demonstrate that DOPart substantially outperforms state-of-the-art online solutions.

I. INTRODUCTION

Computation offloading refers to uploading and executing computationally intensive parts of a computing job on a more powerful remote server. It is a common strategy to overcome the restrictions of resource-constrained mobile devices. For example, mobile cloud/edge computing facilitates computation offloading by providing access to the vast amount of resources hosted by cloud/edge servers. In order to fully exploit the advantages of computational offloading, we need to make intelligent offloading decisions and efficiently utilize the computational resources available in the network.

In this work, we are motivated by a recently emergent class of computation offloading problems concerning the partitioning of inference jobs with deep neural networks (DNNs) between a local mobile device and a remote server. The approximately layer-by-layer structure of an DNN (e.g., ResNet-34 shown in Fig. 1) provides an opportunity to partition an inference job between layers, process the earlier layers at the local device, and offload the later layers to the remote server. Clearly, the total delay in completing the job is reduced if and only if the speed-up due to faster processing at the remote server is greater than the communication delay incurred during offloading.

Since the computation resources on the local device is shared among multiple applications, the local processing delay

is time varying and unknown in advance. Additionally, the offloading delay depends on the state of the communication channel, which also is time varying and unknown in advance especially in the wireless environment. This calls for an *online* algorithm that can dynamically adapt to the uncertainties in the computation and communication environment, in order to minimize the total job completion delay.

There is an extensive body of works in the literature on online partitioning and offloading of computing jobs comprising *independent* tasks (e.g., [1]-[3]). These works exploit the parallel processing of tasks to minimize the total delay. However, in DNNs and other similar applications, the jobs comprise a *dependent* sequence of tasks, or stages, where the execution of each stage requires the completion of the previous stage.

For online offloading of dependent tasks, as detailed further in Section II, most existing works assume the local processing delay and the offloading delay are revealed, possibly estimated by some profiler (e.g., [4] and [5]), when a task arrives. However, in many real-world systems, the available computing resource can vary during the processing of a task, so the processing delay may not be known until after the local execution of a task. Furthermore, reinforcement learning based online solutions (e.g., [6] and [7]) require system specific parameter tuning and generally do not provide performance guarantee. Instead, in this work we propose a lightweight online solution that does not require offline profiling, while still providing provable performance guarantees.

The main contributions of this work are as follows:

- We propose DOPart, an adaptive online algorithm for the partitioning and offloading of a computing job comprising sequentially dependent tasks. It does not require a priori knowledge of communication or local processing latencies. It uses only simple decision rules with low computational complexity, and yet it is highly effective.
- We show that DOPart is optimal in the sense that it achieves the best possible competitive ratio among all deterministic algorithms. In particular, let $\alpha_{\min}$ and $\alpha_{\max}$ be the minimum and maximum ratios between local and remote processing latencies, and consider the non-trivial scenario where $\alpha_{\max} > 1$. If $\alpha_{\min}\alpha_{\max} \geq 1$, DOPart is $\alpha_{\max}$ -competitive, and if $\alpha_{\min}\alpha_{\max} < 1$, it is $\sqrt{\alpha_{\max}/\gamma}$ -competitive where γ is as defined in (15). We prove that these competitive ratios are tight and optimal because they meet the corresponding lower bounds. We also show

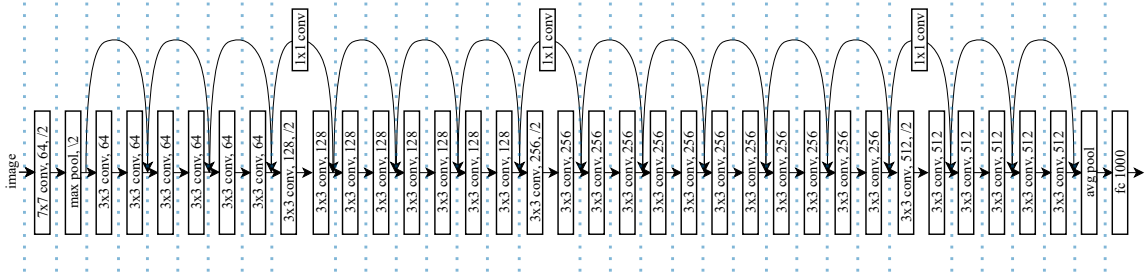


Fig. 1. Architecture of ResNet-34, with sequentially dependent stages indicated by blue dashed lines.

that DOPart offers bounded performance guarantees even when $\alpha_{\min}$ and $\alpha_{\max}$ are not precisely known.

- We experiment with online offloading of DNN inference jobs using AlexNet, VGG-19, and ResNet-34 on the ImageNet dataset over a simulated wireless channel to demonstrate the performance of DOPart and compare it against Neurosurgeon [4], Autodidactic Neurosurgeon [8], and other alternatives. Our numerical results indicate up to 17% reduction in inference time over these state-of-the-art methods.

II. RELATED WORK

A. DNN Partitioning

In the seminar work on DNN partitioning [4], an algorithm named Neurosurgeon was proposed based on system level analysis of the partition points that minimizes the total delay. Subsequently, [9]-[11] considered resource constraints and minimized the total delay in an offline setting. In [5] and [12], offline communication and computation profilers are used to obtain the partitioning points for DNN inference. To reduce the overhead of profiling and parameter storage, [8] proposed a variant of LinUCB for DNN partitioning through online learning.

All of these works utilize some offline profiling or prediction to calculate the optimal partitioning point. The predicted model can become outdated quickly in a dynamic environment where the computation and communication resources fluctuate over time. In our work, we seek an online solution that can automatically adapt to the system dynamics, and we provide performance guarantee even when the future processing and communication latencies are unknown and time varying.

Furthermore, there are also interest studies on dynamic modification of the DNN architecture to minimize the inference delay [13]-[16]. All of these studies substantially differ from our work, since we assume no control over the DNN model's architecture or the job's structure in general, and we cannot compromise on the inference accuracy or the job's intended computational goal.

B. Online Dependent Task Offloading

There is rich literature on the online offloading of dependent tasks to reduce latency. Offloading in single-user single-server systems were considered in [17]-[19], all with general directed acyclic graphs for task dependency. However, all of these works still require profiling and prediction, which as explained

before, do not apply to our problem where the computation and communication latencies can fluctuate unpredictably over time.

Deep reinforcement learning (DRL) techniques have also been proposed for single-user dependent task offloading (e.g., [6] and [7]). DRL techniques require specific hyper parameter tuning tailored to each system setting, and they generally do not provide performance guarantees. Instead, in our work we propose a light-weight solution that is also competitive-ratio optimal.

There are further studies on task offloading in multi-user or multi-server systems, such as [20]-[22]. They mainly address resource contention among devices or applications. Therefore, their solutions are beyond the scope of our problem. Similar to the strategy in [20], our single-user solution may be used as a sub-module in the extension to more complex systems.

III. ONLINE OFFLOADING PROBLEM FORMULATION

A. Sequentially Dependent Stages and Offloading

We consider a job with L sequential stages, with $\mathcal{L} = \{1, 2, \dots, L\}$ denoting the indices of the stages. A local machine and a remote machine collaborate to process this job. The job is initiated at the local machine and may be offloaded to the remote machine at any stage. As explained above, DNN inference offloading with L dependent layers may be viewed as a prime example of this model.

Let P_i^l and P_i^r be the processing delays of stage i on the local and remote machines, respectively. The remote machine usually corresponds to a dedicated processor or virtual machine in the cloud with known speed. Additionally, when the jobs are of DNN inference type, the amount of computation required for each DNN layer solely depends on the number of neurons and weights. Therefore, it is reasonable to assume that the sequence $\{P_i^r\}_{i=1}^L$ is known.

However, due to fluctuation in the available computing power and shared applications at the local machine, $\{P_i^l\}_{i=1}^L$ is random. The only knowledge about P_i^l is its range defined with respect to P_i^r :

$$\alpha_{\min} P_i^r \leq P_i^l \leq \alpha_{\max} P_i^r, \quad (1)$$

where $0 < \alpha_{\min} \leq \alpha_{\max}$. The bounds $\alpha_{\min}$ and $\alpha_{\max}$ can be estimated by computation profiling [4], [10], and in Section V and Section VII we will study the impact of their estimation errors on the competitive ratio and average performance of our solution.

When the job is offloaded at stage i , this means that stages 1 to $i-1$ have been processed locally, and stages i to L will be processed remotely. For mathematical convenience, we further define offloading at stage $L+1$ as not offloading at all. We assume that the remote machine has all programming for the job, so that offloading at stage i involves only transmitting the input data for stage i from the local machine to the remote machine. Note that the input data for stage i is available only after the processing of stage $i-1$ is completed, and the remote machine can start processing stage i only after the input data for this stage is received.

Let d_i be the input data size for stage i . Let $\mathcal{D} = \{d_1, d_2, \dots, d_L, d_{L+1}\}$. Here, d_1 is the size of the input data (e.g., image for classification), and d_{L+1} is the size of the final output (e.g., inferred image label), which in practice is negligibly small and can be returned to the local machine instantly. The communication delay for offloading stage i is $C_i = \frac{d_i}{B_i}$, where B_i is the data rate at the time of offloading. We do not assume any advance knowledge of $\{B_i\}_{i=1}^L$, so that B_i and thus C_i become known only at stage i .

B. Online Delay Minimization Problem

We define an online delay minimization problem instance I as $\{P_i^l, P_i^r, d_i, B_i\}_{i=1}^L$. Given a problem instance, we consider the *total delay*, which is the time to finish all job stages $\mathcal{L}$. If the job is offloaded at stage i , then the total delay, denoted by T_i , includes the processing delay of the first $i-1$ stages on the local machine, the communication delay for offloading, and the processing delay of the last $L-i+1$ stages on the remote machine. Thus, we have

$$T_i = \sum_{k=1}^{i-1} P_k^l + C_i + \sum_{k=i}^L P_k^r. \quad (2)$$

Note that this formula applies also to the case $i = L+1$, which represents no offloading, by defining $C_{L+1} = 0$.

Our optimization objective is to find an optimal offloading point to minimize the total delay. In the offline setting where one has future information of the entire problem instance I , this problem can be easily solved by searching over all possible offloading stages. For problem instance I , the offline optimizer is given by

$$p_{\text{OPT}}(I) = \arg \min_{i \in \mathcal{L} \cup \{L+1\}} T_i. \quad (3)$$

Let $T_{\text{OPT}}(I)$ be the corresponding offline minimum. In practice, one does not have knowledge of the future processing delays P_i^l or communication delays C_i . Despite such uncertainty, we aim to design an online algorithm that provides performance guarantee.

Let $T_{\text{ALG}}(I)$ be the total delay achieved by an online algorithm for problem instance I . The standard performance measurement for an online algorithm is the *competitive ratio*:

$$\rho \triangleq \max_{I \in \mathcal{I}} \frac{T_{\text{ALG}}(I)}{T_{\text{OPT}}(I)}, \quad (4)$$

where $\mathcal{I}$ is the set of all problem instances. When such ρ exists, the online algorithm is called ρ -competitive. The competitive

ratio is the worst-case multiplicative guarantee on the output of the online algorithm when compared with an optimal offline algorithm.

IV. DELAY-OPTIMAL ONLINE PARTITIONING

A. Algorithm Description

Our algorithm is motivated by an adversarial analysis of the possible outcomes of any action. On the one hand, if an online algorithm offloads at any stage $i \in \mathcal{L}$, the adversary will reduce the communication delays of stages $j > i$ and the local processing delays of stages $j \geq i$ to the minimum, to incur a worst possible outcome for the algorithm's decision. On the other hand, if the online algorithm does not offload at stage i , the adversary will increase the communication delays of the stages $j > i$ and the local processing delays of stages $j \geq i$ to the maximum. An effective solution needs to balance these two extremes.

Suppose an online algorithm is at stage i deciding whether to offload the remainder of the job starting from this stage. This means that all stages $j \leq i-1$ have already been processed on the local machine, incurring known delay $\sum_{k=1}^{i-1} P_k^l$. If the job is offloaded at stage i , then the total processing delay will be

$$\bar{T}_i = \sum_{k=1}^{i-1} P_k^l + \sum_{k=i}^L P_k^r, \quad (5)$$

which is a known quantity. If instead the job is offloaded at any later stage $j > i$, the total processing delay will be $\sum_{k=1}^{i-1} P_k^l + \sum_{k=i}^{j-1} P_k^l + \sum_{k=j}^L P_k^r$. We note that $\sum_{k=i}^{j-1} P_k^l$ is unknown at this time, but it is lower bounded by $\alpha_{\min} \sum_{k=i}^{j-1} P_k^r$. Considering all possible $i+1 \leq j \leq L+1$, it is easy to see that, at stage i , the minimum possible total processing delay is

$$T_i^{\min} = \sum_{k=1}^{i-1} P_k^l + \alpha_{\min} P_i^r + \min\{\alpha_{\min}, 1\} \sum_{k=i+1}^L P_k^r. \quad (6)$$

Similarly, while at stage i , the maximum possible total processing delay is

$$T_i^{\max} = \sum_{k=1}^{i-1} P_k^l + \max\{\alpha_{\max}, 1\} \sum_{k=i}^L P_k^r. \quad (7)$$

We also need to account for the offloading communication delay. In DOPart, we use the geometric mean of the lower and upper bounds above as the threshold for our offloading decision. At stage i , we observe the current communication data rate B_i to obtain the communication delay C_i . We offload the job at this stage if and only if

$$C_i + \bar{T}_i \leq \sqrt{T_i^{\min} T_i^{\max}}. \quad (8)$$

We remark here that (8) is similar in form to the optimal time-series search [23]. However, there are two main differences between our design and the standard time-series search. First, the lower and upper bounds of the processing time in our problem varies over time. Second, in addition to the sequence

Algorithm 1: Delay-optimal Online Partitioning (DOPart)

Input: $\mathcal{L}$, $\{d_i\}_{i=1}^L$, $\{P_i^r\}_{i=1}^L$, $\alpha_{\min}$, $\alpha_{\max}$

```

1  $\mathcal{F} \leftarrow \emptyset$ 
2 for  $i \leftarrow 1$  to  $L$  do
3   Compute  $T_i^{\min}$ ,  $T_i^{\max}$ , and  $\bar{T}_i$ 
4    $B_i \leftarrow$  Observe current data rate
5    $C_i \leftarrow d_i/B_i$ 
6   if  $C_i + \bar{T}_i \leq \sqrt{T_i^{\min}T_i^{\max}}$  then
7     Transmit input data for stage  $i$ 
8     Process stages  $\mathcal{L} \setminus \mathcal{F}$  on the remote machine
9     Exit
10  end
11  Process stage  $i$  on the local machine
12   $P_i^l \leftarrow$  Observe local processing delay of stage  $i$ 
13   $\mathcal{F} \leftarrow \mathcal{F} \cup i$ 
14 end

```

of processing times, here we need to take an additional factor, the communication delay, into consideration. Note that only the left-hand side of (8) contains the communication delay, while the right-hand side is based on the processing delay only. We will show in Section IV-B that this unique design leads to the best possible competitive ratio for our optimization problem. The pseudocode for the proposed DOPart algorithm is given in Algorithm 1.

Complexity: Since DOPart processes the stages sequentially, the time complexity of DOPart is $\mathcal{O}(L)$, i.e., linear in the number of stages. We note that this is a substantial reduction from existing methods that use deep reinforcement learning, which do not provide any performance guarantee.

B. Competitiveness Analysis

We derive the competitive ratio for DOPart under three different cases: (1) $\alpha_{\max} \leq 1$, (2) $\alpha_{\max} > 1$ and $\alpha_{\min}\alpha_{\max} < 1$, and (3) $\alpha_{\max} > 1$ and $\alpha_{\min}\alpha_{\max} \geq 1$. For all cases, we will additionally show that DOPart is competitive-ratio optimal, i.e., there does not exist another deterministic online algorithm that achieves a lower competitive ratio. This also directly implies that our derived competitive ratios are tight.

For notational convenience, for $i \in \mathcal{L} \cup \{L+1\}$ we define $T_{-i}^l = \sum_{k=1}^{i-1} P_k^l$, $T_{+i}^l = \sum_{k=i}^L P_k^l$, and $T^l = T_{-i}^l + T_{+i}^l$, with the last being the total local processing delay if all stages are processed locally. We also define T_{-i}^r, T_{+i}^r , and T^r similarly, for the remote processing delays.

Before presenting the main results, we require the following two lemmas, whose proofs are omitted due to the page limit.

Lemma 1. For any $\alpha_{\min} \leq \alpha_{\max}$, we have $T_{-i}^l + \alpha_{\max}T_{+i}^r \leq T_{-k}^l + \alpha_{\max}T_{+k}^r$ for all $i > k$.

Lemma 2. For any $\alpha_{\min} \leq \alpha_{\max}$, we have $T_{-i}^l + \alpha_{\min}P_i^r + \min\{\alpha_{\min}, 1\}T_{+(i+1)}^r \leq T_{-k}^l + \min\{\alpha_{\min}, 1\}T_{+k}^r$ for all $i < k$.

Furthermore, since $\alpha_{\min} \geq \min\{\alpha_{\min}, 1\}$, the corollary below directly follows from Lemma 2:

Corollary 3. For any $\alpha_{\min}$ and $\alpha_{\max}$, we have $T_{-i}^l + \alpha_{\min}P_i^r + \min\{\alpha_{\min}, 1\}T_{+(i+1)}^r \leq T_{-k}^l + \alpha_{\min}P_k^r + \min\{\alpha_{\min}, 1\}T_{+(k+1)}^r$ for all $i < k$.

1) *Case (1): $\alpha_{\max} \leq 1$:* In the simplest case of $\alpha_{\max} \leq 1$, i.e., the processing delay of the local machine is always less than that on the remote machine, DOPart is 1-competitive. This means that it performs as well as the optimal offline algorithm, and clearly no other algorithm can do better.

Theorem 4. If $\alpha_{\max} \leq 1$, DOPart is 1-competitive.

Proof. When $\alpha_{\max} < 1$, since $\alpha_{\min} \leq \alpha_{\max}$, we have $\alpha_{\min}\alpha_{\max} < 1$. Based on (8), DOPart never offloads i.e., it processes all the stages locally. Meanwhile, the best offline strategy in hindsight is to never offload because $C_i \geq 0$, $\forall i \in \mathcal{L}$, and $P_i^l < P_i^r$, $\forall i \in \mathcal{L}$. Therefore, DOPart behaves the same as the optimal offline algorithm in this case. $\square$

2) *Case (2): $\alpha_{\max} > 1$ and $\alpha_{\min}\alpha_{\max} < 1$:* Unlike the previous trivial case, when $\alpha_{\max} > 1$, there can be potential benefit in offloading the job to the remote machine. In this case, since we also have $\alpha_{\min} < 1/\alpha_{\max} < 1$, the local machine can be faster than the remote machine. Therefore, DOPart is required to be more conservative with the offloading decision.

First, since in this case $\min\{\alpha_{\min}, 1\} = \alpha_{\min}$, we have a further corollary from Lemma 2:

Corollary 5. When $\alpha_{\min} < 1$, $T_{-i}^l + \alpha_{\min}T_i^r \leq T_{-k}^l + \alpha_{\min}T_{+k}^r$ for all $i < k$.

We also require the following lemma, whose proof is omitted due to the page limit.

Lemma 6. When $\alpha_{\max} > 1$ and $\alpha_{\min}\alpha_{\max} < 1$, if DOPart offloads at stage i then $\alpha_{\min} + \alpha_{\max} > 2$ and $T_{-i}^l \geq \phi T_{+i}^r$, where

$$\phi = \left(\frac{1 - \alpha_{\min}\alpha_{\max}}{\alpha_{\min} + \alpha_{\max} - 2} \right). \quad (9)$$

Theorem 7. If $\alpha_{\max} > 1$ and $\alpha_{\min}\alpha_{\max} < 1$,

- (i) No deterministic online algorithm can do better than $\alpha_{\max}$ -competitive, and
- (ii) DOPart is $\alpha_{\max}$ -competitive.

Proof. First we prove part (i). To show this we take an adversarial approach. The adversary first sets $C_1 = 0$. If any deterministic online algorithm ALG offloads at stage 1, the total delay $T_{\text{ALG}} = T^r$. The adversary then sets the local processing latencies $P_k^l = \alpha_{\min}P_k^r \forall k \in \{1, \dots, L\}$. Since $\alpha_{\min} < 1$, the total delay of optimal offline algorithm $T_{\text{OPT}} = \alpha_{\min}T^r$. Thus in this case $T_{\text{ALG}}/T_{\text{OPT}} \geq 1/\alpha_{\min}$. If ALG does not offload at stage 1, the adversary sets the local processing latencies $P_k^l = \alpha_{\max}P_k^r, \forall k \in \{1, \dots, L\}$, and $C_k = (\alpha_{\max} - 1)T_{+k}^r, \forall k \in \{2, \dots, L\}$. In this case the total delay $T_{\text{ALG}} = \alpha_{\max}T^r$. Since $\alpha_{\max} > 1$,

$T_{\text{OPT}} = C_1 + T^r = T^r$. Then in this case $T_{\text{ALG}}/T_{\text{OPT}} \geq \alpha_{\max}$. Because $1/\alpha_{\min} > \alpha_{\max}$, we have

$$\rho_{\text{ALG}} \geq \min \left\{ \alpha_{\max}, \frac{1}{\alpha_{\min}} \right\} = \alpha_{\max}.$$

We now prove part (ii). If $\alpha_{\min} + \alpha_{\max} \leq 2$, from Lemma 6, DOPart does not offload at any stage. Then the total delay is $T_{\text{DOPart}} = T^l$. Suppose the optimal offline algorithm OPT that offloads at layer k . Its total delay is $T_{\text{OPT}} = T_{-k}^l + C_k + T_{+k}^r \geq T_{-k}^l + T_{+k}^r$, with equality when $C_k = 0$. The competitive ratio is then given by

$$\begin{aligned} \rho_{\text{DOPart}} &= \max_{I \in \mathcal{I}} \left\{ \frac{T^l}{T_{-k}^l + T_{+k}^r} \right\} = \max_{I \in \mathcal{I}} \left\{ \frac{T_{-k}^l + T_{+k}^l}{T_{-k}^l + T_{+k}^r} \right\} \\ &\stackrel{(a)}{=} \max_{I \in \mathcal{I}} \left\{ \frac{T_{-k}^l + \alpha_{\max} T_{+k}^r}{T_{-k}^l + T_{+k}^r} \right\} \stackrel{(b)}{=} \alpha_{\max}. \end{aligned} \quad (10)$$

Here equality (a) is because $T_{+k}^l \leq \alpha_{\max} T_{+k}^r$, and equality (b) is because $\alpha_{\max} > 1$.

The case where $\alpha_{\min} + \alpha_{\max} > 2$ is more complicated. Define $q = \min\{s \in \mathcal{L} \cup \{L+1\} | T_{-s}^l \geq \phi T_{+s}^r\}$. From Lemma 6 we know DOPart offloads at stage i such that $T_{-i}^l \geq \phi T_{+i}^r$, so we have $i \geq q$. The total delay of DOPart is given by

$$\begin{aligned} T_{\text{DOPart}} &= T_{-i}^l + C_i + T_{+i}^r \\ &\stackrel{(a)}{\leq} T_{-i}^l + \sqrt{T_i^{\min} T_i^{\max}} - \bar{T}_i + T_{+i}^r \\ &\stackrel{(b)}{=} \sqrt{(T_{-i}^l + \alpha_{\min} T_{+i}^r)(T_{-i}^l + \alpha_{\max} T_{+i}^r)}. \end{aligned} \quad (11)$$

Here, inequality (a) is due to the offloading threshold in (8). Equality (b) is because $\min\{\alpha_{\min}, 1\} = \alpha_{\min}$, so that $\alpha_{\min} P_i^r + \min\{\alpha_{\min}, 1\} T_{+(i+1)}^r = \alpha_{\min} T_{+i}^r$.

Suppose OPT offloads at stage $k \in \mathcal{L} \cup \{L+1\}$. Its latency is

$$T_{\text{OPT}} = T_{-k}^l + C_k + T_{+k}^r \geq T_{-k}^l + T_{+k}^r. \quad (12)$$

Then from (11) and (12) we have

$$\frac{T_{\text{DOPart}}^2}{T_{\text{OPT}}^2} \leq \max_{I \in \mathcal{I}} \left\{ \frac{(T_{-i}^l + \alpha_{\min} T_{+i}^r)(T_{-i}^l + \alpha_{\max} T_{+i}^r)}{(T_{-k}^l + T_{+k}^r)^2} \right\} \quad (13)$$

If $k < q$, from (13) we have

$$\begin{aligned} \frac{T_{\text{DOPart}}^2}{T_{\text{OPT}}^2} &\stackrel{(a)}{\leq} \max_{I \in \mathcal{I}} \left\{ \frac{T_{-k}^l + T_{-i}^l - T_{-k}^l + \alpha_{\min} T_{+i}^r}{T_{-k}^l + T_{+k}^r} \right\} \\ &\stackrel{(b)}{\leq} \max_{I \in \mathcal{I}} \left\{ \frac{T_{-k}^l + \alpha_{\max}(T_{+k}^r - T_{+i}^r) + \alpha_{\min} T_{+i}^r}{T_{-k}^l + T_{+k}^r} \right\} \\ &\stackrel{(c)}{\leq} \max_{I \in \mathcal{I}} \left\{ \frac{T_{-k}^l + \alpha_{\max}(T_{+k}^r - T_{+i}^r) + \alpha_{\max} T_{+i}^r}{T_{-k}^l + T_{+k}^r} \right\} \\ &\stackrel{(d)}{\leq} \max_{I \in \mathcal{I}} \left\{ \left(\frac{T_{-k}^l + \alpha_{\max} T_{+k}^r}{T_{-k}^l + T_{+k}^r} \right)^2 \right\} \leq \alpha_{\max}^2 \end{aligned}$$

Here, equality (a) is because T_{-i}^l can be split into T_{-k}^l and $T_{-i}^l - T_{-k}^l$ since $k \leq i$. Inequality (b) is because $T_{-i}^l - T_{-k}^l \leq \alpha_{\max}(T_{+k}^r - T_{+i}^r)$ from Lemma 1. Inequality (c) is because $(\alpha_{\min} - \alpha_{\max})T_{+i}^r \leq 0$. Inequality (d) is because $\alpha_{\max} > 1$.

If $q \leq k < i$, we have

$$\begin{aligned} T_{\text{OPT}} &= T_{-k}^l + C_k + T_{+k}^r \\ &\stackrel{(a)}{\geq} T_{-k}^l + \sqrt{T_k^{\min} T_k^{\max}} - \bar{T}_k + T_{+k}^r \\ &\stackrel{(b)}{=} \sqrt{(T_{-k}^l + \alpha_{\min} T_{+k}^r)(T_{-k}^l + \alpha_{\max} T_{+k}^r)}. \end{aligned} \quad (14)$$

Here, inequality (a) is because $k < i$, which means that DOPart does not offload at k , so from (8) we have $C_k \geq \sqrt{T_k^{\min} T_k^{\max}} - \bar{T}_k$. Equality (b) is because of (5). From (11) and (14) we have

$$\begin{aligned} \frac{T_{\text{DOPart}}^2}{T_{\text{OPT}}^2} &\leq \max_{I \in \mathcal{I}} \left\{ \frac{(T_{-i}^l + \alpha_{\min} T_{+i}^r)(T_{-i}^l + \alpha_{\max} T_{+i}^r)}{(T_{-k}^l + \alpha_{\min} T_{+k}^r)(T_{-k}^l + \alpha_{\max} T_{+k}^r)} \right\} \\ &\stackrel{(a)}{\leq} \max_{I \in \mathcal{I}} \left\{ \frac{T_{-k}^l + \alpha_{\max}(T_{+k}^r - T_{+i}^r) + \alpha_{\min} T_{+i}^r}{T_{-k}^l + \alpha_{\min} T_{+k}^r} \right\} \\ &\stackrel{(b)}{\leq} \max_{I \in \mathcal{I}} \left\{ \frac{T_{-k}^l + \alpha_{\max}(T_{+k}^r - T_{+i}^r) + \alpha_{\max} T_{+i}^r}{T_{-k}^l + \alpha_{\max} T_{+k}^r} \right\} \\ &\stackrel{(c)}{\leq} \max_{I \in \mathcal{I}} \left\{ \frac{\phi T_{+k}^r + \alpha_{\max} T_{+k}^r}{\phi T_{+k}^r + \alpha_{\min} T_{+k}^r} \right\} \\ &= \left(\frac{\alpha_{\max} - 1}{1 - \alpha_{\min}} \right)^2 \stackrel{(d)}{<} \alpha_{\max}^2 \end{aligned}$$

Here, inequality (a) is because T_{-i}^l can be split into T_{-k}^l and $T_{-i}^l - T_{-k}^l$, and $T_{-i}^l - T_{-k}^l \leq \alpha_{\max}(T_{+k}^r - T_{+i}^r)$ from Lemma 1. Inequality (b) is because $(\alpha_{\min} - \alpha_{\max})T_{+i}^r \leq 0$ since $\alpha_{\min} \leq \alpha_{\max}$. Inequality (c) is because of Lemma 6 and $\alpha_{\max} > \alpha_{\min}$. Inequality (d) is because $\alpha_{\min} < 1/\alpha_{\max}$.

If $k = i$, $T_{\text{DOPart}} = T_{\text{OPT}}$. Then we have $\frac{T_{\text{DOPart}}}{T_{\text{OPT}}} = 1 < \alpha_{\max}$.

If $k > i$, then from (13) we have

$$\begin{aligned} \frac{T_{\text{DOPart}}^2}{T_{\text{OPT}}^2} &\stackrel{(a)}{<} \max_{I \in \mathcal{I}} \left\{ \frac{(T_{-i}^l + \alpha_{\min} T_{+i}^r)(T_{-i}^l + \alpha_{\max} T_{+i}^r)}{(T_{-i}^l + \alpha_{\min} T_{+i}^r)^2} \right\} \\ &\stackrel{(b)}{=} \frac{\phi T_{+i}^r + \alpha_{\max} T_{+i}^r}{\phi T_{+i}^r + \alpha_{\min} T_{+i}^r} = \left(\frac{\alpha_{\max} - 1}{1 - \alpha_{\min}} \right)^2 \stackrel{(c)}{<} \alpha_{\max}^2 \end{aligned}$$

Here, inequality (a) is from $T_{-k}^l + T_{+k}^r > T_{-k}^l + \alpha_{\min} T_{+k}^r$ and Corollary 5. Equality (b) is from Lemma 6 and $\alpha_{\max} > \alpha_{\min}$. Inequality (c) is because $\alpha_{\min} < 1/\alpha_{\max}$.

Summarizing the above, we conclude that we also have $\rho_{\text{DOPart}} = \alpha_{\max}$ for the case $\alpha_{\min} + \alpha_{\max} > 2$. $\square$

3) Case (3): $\alpha_{\max} > 1$ and $\alpha_{\min} \alpha_{\max} \geq 1$: This case represents an environment where offloading to the remote machine generally offers a higher speedup in comparison with case (2). Since there is a higher chance for delay reduction in offloading, our algorithm is allowed to be more liberal with the offloading threshold.

Theorem 8. If $\alpha_{\max} > 1$ and $\alpha_{\max}\alpha_{\min} \geq 1$,

(i) No deterministic online algorithm can do better than $\sqrt{\alpha_{\max}/\gamma}$ -competitive where

$$\gamma = \min\{\alpha_{\min}, 1\} + (\alpha_{\min} - \min\{\alpha_{\min}, 1\})(P_1^r/T^r), \quad (15)$$

(ii) DOPart is $\sqrt{\alpha_{\max}/\gamma}$ -competitive, and

(iii) A simplified form follows that DOPart is at least $\sqrt{\alpha_{\max}/\min\{\alpha_{\min}, 1\}}$ -competitive.

Proof. We proof parts (i) and (ii) similarly to the corresponding parts of Theorem 7, except in this case $\alpha_{\min} \geq 1/\alpha_{\max}$ and we also take into account $\alpha_{\min} > 1$. Part (iii) follows directly from part (ii) since $(\alpha_{\min} - \min\{\alpha_{\min}, 1\})(P_1^r/T^r) \geq 0$. $\square$

V. COMPETITIVE RATIO WITH ESTIMATION ERRORS

We extend our competitive analysis to the scenario where we only have access to estimates of $\alpha_{\min}$ and $\alpha_{\max}$, defined as $\hat{\alpha}_{\min} = \alpha_{\min}(1 + \epsilon_1)$ and $\hat{\alpha}_{\max} = \alpha_{\max}(1 + \epsilon_2)$, with errors $\epsilon_1, \epsilon_2 \in \mathbb{R}$. Without loss of generality, we assume that the estimation errors are small enough so that $\hat{\alpha}_{\min} \leq \hat{\alpha}_{\max}$.

The following theorems indicate that even with imprecise estimation of $\alpha_{\min}$ and $\alpha_{\max}$, DOPart provides bounded performance guarantees. Our competitive ratio analysis in this more complex scenario uses similar techniques as in Section IV-B, with additional finesse required to account for the estimation errors and handle different cases where ϵ_1 and ϵ_2 are positive or negative. The proof details are omitted due to the page limit.

Theorem 9. If $\hat{\alpha}_{\max} \leq 1$, $\rho_{\text{DOPart}} = \max\{1, \alpha_{\max}\}$.

Theorem 10. If $\hat{\alpha}_{\max} > 1$ and $\hat{\alpha}_{\min}\hat{\alpha}_{\max} < 1$,

$$\rho_{\text{DOPart}}(\epsilon_1, \epsilon_2) = \alpha_{\max} \max\{1, 1 + \epsilon_2, (1 + \epsilon_1)(1 + \epsilon_2)\}.$$

Theorem 11. If $\hat{\alpha}_{\max} > 1$ and $\hat{\alpha}_{\min}\hat{\alpha}_{\max} \geq 1$,

$$\rho_{\text{DOPart}}(\epsilon_1, \epsilon_2) = \sqrt{\frac{\alpha_{\max}}{\min\{\alpha_{\min}, 1\}} \frac{1}{1 - |\epsilon_1|} \frac{1}{1 - |\epsilon_2|}}.$$

The above results give insight into how the performance of DOPart is impacted by ϵ_1 and ϵ_2 . Comparing Theorems 10 and 11 with their counter parts Theorems 7 and 8, and applying the Taylor series expansion on the extra factors, we observe that the competitiveness of DOPart deteriorates approximately linearly in ϵ_1 and ϵ_2 . Therefore, DOPart is robust against small to moderate estimation errors.

The preceding sections establish the performance of deterministic online partitioning algorithms. In particular, DOPart achieves the optimal deterministic competitive ratio, and the same analysis quantifies how this ratio changes under estimation errors. These results leave open a natural question: can randomization improve the worst-case guarantee when performance is measured in expectation over the algorithm's internal randomness? We answer this question next by developing a randomized thresholding rule and then proving an asymptotically tight guarantee for its static-threshold form.

Algorithm 2: Randomized Delay-optimal Online Partitioning (DOPart-R)

Input: $\mathcal{L}, \{d_i\}_{i=1}^L, \{P_i^r\}_{i=1}^L, \alpha_{\min}, \alpha_{\max}$

- 1 $\mathcal{F} \leftarrow \emptyset$
- 2 Draw $U \sim \text{Unif}[0, 1]$
- 3 **for** $i \leftarrow 1$ **to** L **do**
- 4 Compute $\bar{T}_i, T_i^{\min}, T_i^{\max}, \alpha_i^{\min}, \alpha_i^{\max}, \epsilon_i$, and ρ_i
- 5 $\hat{V}_i \leftarrow \alpha_i^{\max} - (\alpha_i^{\max} - \alpha_i^{\min})e^{-U/\rho_i}$
- 6 $B_i \leftarrow$ Observe current data rate
- 7 $C_i \leftarrow d_i/B_i$
- 8 $T_i \leftarrow \sum_{k=1}^{i-1} P_k^l + C_i + \sum_{k=i}^L P_k^r$
- 9 **if** $T_i \leq \hat{V}_i \bar{T}_i$ **then**
- 10 Transmit input data for stage i
- 11 Process stages $\mathcal{L} \setminus \mathcal{F}$ on the remote machine
- 12 Exit
- 13 **end**
- 14 Process stage i on the local machine
- 15 $P_i^l \leftarrow$ Observe local processing delay of stage i
- 16 $\mathcal{F} \leftarrow \mathcal{F} \cup i$
- 17 **end**

VI. RANDOMIZED ALGORITHM

We now move from deterministic thresholding to randomized thresholding. For a fixed instance, with a fixed delay sequence, the offline algorithm finds the best threshold for the delay sequence, while the online algorithm samples a threshold before making offloading decisions. The performance metric is therefore the expected delay of the algorithm, where the expectation is taken only over this sampled threshold.

A. Algorithm Description

Randomized Delay-optimal Online Partitioning (DOPart-R), shown in Algorithm 2, is the randomized counterpart of DOPart. It keeps the same online information pattern: at stage i , the local processing delays of stages $1, \dots, i-1$ are known, while the current communication delay is revealed only after observing the data rate B_i . The difference is that DOPart-R does not compare the immediate offloading delay with a deterministic threshold. Instead, it draws one random percentile at the beginning and maps this percentile to a stage-dependent threshold distribution at each later stage.

At stage i , DOPart-R treats the unprocessed stages as a suffix problem. Define

$$\begin{aligned} T_i^{\max} &= T_{-i}^l + \max\{\alpha_{\max}, 1\}T_{+i}^r, \\ T_i^{\min} &= T_{-i}^l + \min\{\alpha_{\min}, 1\}T_{+i}^r, \\ \bar{T}_i &= T_{-i}^l + T_{+i}^r. \end{aligned}$$

Here, $T_i^{\max}$ and $T_i^{\min}$ are upper and lower bounds on the processing delay of any suffix decision, and $\bar{T}_i$ is the suffix baseline obtained by processing the completed prefix locally

and the remaining suffix remotely. We normalize these bounds as

$$\alpha_i^{\max} = \frac{T_i^{\max}}{\bar{T}_i}, \quad \alpha_i^{\min} = \frac{T_i^{\min}}{\bar{T}_i}.$$

For each stage i , define

$$f_V^{\alpha_i^{\min}, \alpha_i^{\max}}(v) = \frac{\alpha_i^{\max}}{\alpha_i^{\max} - \epsilon_i} \frac{1}{\alpha_i^{\max} - v}, \quad v \in [\alpha_i^{\min}, \alpha_i^{\max} - \epsilon_i],$$

where

$$\epsilon_i = -\alpha_i^{\max} W\left(\frac{1 - \alpha_i^{\max}/\alpha_i^{\min}}{e\alpha_i^{\max}/\alpha_i^{\min}}\right),$$

and $W(\cdot)$ is the Lambert W function. We also define

$$\rho_i = \frac{\alpha_i^{\max}}{\alpha_i^{\max} - \epsilon_i}.$$

The value of ϵ_i is chosen so that

$$\int_{\alpha_i^{\min}}^{\alpha_i^{\max} - \epsilon_i} f_V^{\alpha_i^{\min}, \alpha_i^{\max}}(v) dv = 1,$$

so $f_V^{\alpha_i^{\min}, \alpha_i^{\max}}$ is a valid density on $[\alpha_i^{\min}, \alpha_i^{\max} - \epsilon_i]$.

Let F_i denote the CDF corresponding to the stage- i density:

$$F_i(v) = \begin{cases} 0, & v \leq \alpha_i^{\min}, \\ \rho_i \ln\left(\frac{\alpha_i^{\max} - \alpha_i^{\min}}{\alpha_i^{\max} - v}\right), & \alpha_i^{\min} < v < \alpha_i^{\max} - \epsilon_i, \\ 1, & v \geq \alpha_i^{\max} - \epsilon_i. \end{cases}$$

Instead of drawing independent thresholds at different stages, DOPart-R draws one common percentile $U \sim \text{Unif}[0, 1]$ and maps it to the current suffix distribution:

$$\hat{V}_i = F_i^{-1}(U) = \alpha_i^{\max} - (\alpha_i^{\max} - \alpha_i^{\min})e^{-U/\rho_i}.$$

Thus each $\hat{V}_i$ has marginal density $f_V^{\alpha_i^{\min}, \alpha_i^{\max}}$, while the thresholds across stages are coupled through the common percentile U . This is equivalent to drawing $\hat{V}_1$ from the first-stage distribution and preserving its percentile at all later stages, but it is simpler to state directly in terms of U . DOPart-R then observes B_i , computes $C_i = d_i/B_i$, and evaluates the total delay $T_i = T_{-i}^l + C_i + T_{+i}^r$ that would result from offloading at stage i . It offloads if

$$T_i \leq \hat{V}_i \bar{T}_i.$$

Otherwise, it processes stage i locally, observes P_i^l , and repeats the same percentile-matching rule on the next suffix. If no stage satisfies the offloading condition, the job is completed locally. Since DOPart-R scans the stages sequentially and performs only constant-time computations at each stage, its time complexity is $\mathcal{O}(L)$. Its performance is measured by the expected total delay, where the expectation is taken over the single sampled percentile.

B. Competitive Analysis

Definition 12. The helper DSR draws one threshold $\hat{V}_1 \sim f_V^{\alpha_1^{\min}, \alpha_1^{\max}}$. It then offloads at the first stage $j \in \{1, \dots, L\}$ satisfying

$$T_j \leq \hat{V}_1 \bar{T}_1.$$

If no such stage exists, it processes all stages locally.

Let

$$T_{\text{OPT}} = \min_{j \in \mathcal{L} \cup \{L+1\}} T_j, \quad \mu_1 = \frac{T_{\text{OPT}}}{\bar{T}_1}.$$

Then $\alpha_1^{\min} \leq \mu_1 \leq \alpha_1^{\max}$.

Lemma 13. On every fixed instance,

$$\mathbb{E}[T_{\text{DSR}}] \leq \rho_1 T_{\text{OPT}}.$$

Proof. If $\hat{V}_1 < \mu_1$, then no stage j satisfies $T_j \leq \hat{V}_1 \bar{T}_1$, so DSR does not offload. In this case,

$$T_{\text{DSR}} = T_{L+1} \leq T_1^{\max} = \alpha_1^{\max} \bar{T}_1.$$

If $\hat{V}_1 \geq \mu_1$, then the optimal offloading stage satisfies $T_{\text{OPT}} \leq \hat{V}_1 \bar{T}_1$, so DSR offloads at some stage j with

$$T_{\text{DSR}} = T_j \leq \hat{V}_1 \bar{T}_1.$$

First consider $\mu_1 \in [\alpha_1^{\min}, \alpha_1^{\max} - \epsilon_1]$. Using the two cases above,

$$\begin{aligned} \mathbb{E}[T_{\text{DSR}}] &\leq \alpha_1^{\max} \bar{T}_1 \int_{\alpha_1^{\min}}^{\mu_1} \rho_1 \frac{1}{\alpha_1^{\max} - v} dv \\ &\quad + \int_{\mu_1}^{\alpha_1^{\max} - \epsilon_1} v \bar{T}_1 \rho_1 \frac{1}{\alpha_1^{\max} - v} dv \\ &= \rho_1 \mu_1 \bar{T}_1 = \rho_1 T_{\text{OPT}}. \end{aligned}$$

Next consider $\mu_1 \in [\alpha_1^{\max} - \epsilon_1, \alpha_1^{\max}]$. Then

$$\mathbb{E}[T_{\text{DSR}}] \leq \alpha_1^{\max} \bar{T}_1 \leq \frac{\alpha_1^{\max}}{\alpha_1^{\max} - \epsilon_1} \mu_1 \bar{T}_1 = \rho_1 T_{\text{OPT}}.$$

Hence $\mathbb{E}[T_{\text{DSR}}] \leq \rho_1 T_{\text{OPT}}$ in both cases. $\square$

Lemma 14. In DOPart-R the stage- j threshold $\hat{V}_j$ has marginal density $f_V^{\alpha_j^{\min}, \alpha_j^{\max}}$.

Proof. Let F_j be the CDF of $f_V^{\alpha_j^{\min}, \alpha_j^{\max}}$. DOPart-R draws $U \sim \text{Unif}[0, 1]$ and sets $\hat{V}_j = F_j^{-1}(U)$, namely

$$\hat{V}_j = \alpha_j^{\max} - (\alpha_j^{\max} - \alpha_j^{\min})e^{-U/\rho_j}.$$

Therefore, for every v in the support of $f_V^{\alpha_j^{\min}, \alpha_j^{\max}}$,

$$\Pr\{\hat{V}_j \leq v\} = \Pr\{U \leq F_j(v)\} = F_j(v).$$

Thus $\hat{V}_j$ has density $f_V^{\alpha_j^{\min}, \alpha_j^{\max}}$. $\square$

Lemma 15. Fix an instance and an optimal stage m , so $T_m = T_{\text{OPT}}$. For $k \leq m$, define

$$Q_k(u) = F_k^{-1}(u) \bar{T}_k = T_k^{\max} - (T_k^{\max} - T_k^{\min})e^{-u/\rho_k},$$

$$h_k = F_k\left(\frac{T_m}{\bar{T}_k}\right), \quad H_k(u; T_m) = \begin{cases} T_k^{\max}, & u < h_k, \\ Q_k(u), & u \geq h_k. \end{cases}$$

Then, for every $k < m$ and every percentile cutoff $z \in [0, 1]$,

$$\int_0^z H_{k+1}(u; T_m) du \leq \int_0^z H_k(u; T_m) du.$$

Proof. Let $I_k(z) = \int_0^z H_k(u; T_m) du$. Direct integration gives

$$I_k(z) = zT_k^{\max}, \quad z \leq h_k,$$

and

$$I_k(z) = zT_k^{\max} - \rho_k(T_k^{\max} - T_m) + \rho_k(T_k^{\max} - T_k^{\min})e^{-z/\rho_k}, \quad z > h_k.$$

This expression is the DSR envelope integral applied to the stage- k suffix interval $[T_k^{\min}, T_k^{\max}]$. A direct derivative check, using

$$\rho_k \ln\left(\frac{T_k^{\max} - T_k^{\min}}{\epsilon_k \bar{T}_k}\right) = 1, \quad \rho_k = \frac{T_k^{\max}}{T_k^{\max} - \epsilon_k \bar{T}_k},$$

shows that $I_k(z)$ cannot increase when the suffix interval shrinks around the fixed value T_m . Since

$$T_k^{\min} \leq T_{k+1}^{\min} \leq T_m \leq T_{k+1}^{\max} \leq T_k^{\max},$$

we get $I_{k+1}(z) \leq I_k(z)$ for every $z \in [0, 1]$. $\square$

Theorem 16. *DOPart-R algorithm is ρ_1 -competitive in expectation. That is, for every fixed instance,*

$$\mathbb{E}[T_{\text{DOPart-R}}] \leq \rho_1 T_{\text{OPT}},$$

where the expectation is taken only over the sampled percentile.

Proof. Fix an instance and let m be an optimal stage, so $T_m = T_{\text{OPT}}$. Use the notation Q_i, h_i, H_i from Lemma 15. For a fixed percentile u , DOPart-R offloads at the first stage i satisfying $T_i \leq Q_i(u)$.

Let R_i be the set of percentiles that reach stage i . Since each $Q_i(u)$ is increasing in u , each R_i is a prefix interval $[0, z_i]$. At any stage $i < m$, the percentiles in $R_i \setminus R_{i+1}$ stop at stage i . For such a percentile,

$$T_i \leq Q_i(u).$$

Since $T_i \geq T_m$, accepting T_i implies $u \geq h_i$, and therefore $H_i(u; T_m) = Q_i(u)$. Hence

$$T_i \leq H_i(u; T_m), \quad u \in R_i \setminus R_{i+1}.$$

For the surviving prefix $R_{i+1} = [0, z_{i+1}]$, Lemma 15 gives

$$\int_{R_{i+1}} H_{i+1}(u; T_m) du \leq \int_{R_{i+1}} H_i(u; T_m) du.$$

Therefore, replacing the stopped part by actual cost and passing the surviving part to the next envelope cannot increase the total charge. Iterating this one-step argument from stage 1 to stage m yields

$$\int_0^1 T_{\text{DOPart-R}}(u) du \leq \int_0^1 H_1(u; T_m) du.$$

It remains only to justify the last stage in the iteration. If a percentile reaches stage m and $u \geq h_m$, then DOPart-R accepts stage m and pays $T_m \leq Q_m(u) = H_m(u; T_m)$. If $u < h_m$, then the envelope value is $H_m(u; T_m) = T_m^{\max}$, which bounds any later accepted stage and also the no-offloading delay. Thus all percentiles that reach m are covered by H_m .

Finally, the initial envelope is exactly the DSR envelope. If $T_m \leq T_1^{\max} - \epsilon_1 \bar{T}_1$, the calculation in Lemma 13 gives

$$\int_0^1 H_1(u; T_m) du = \rho_1 T_m.$$

If $T_m > T_1^{\max} - \epsilon_1 \bar{T}_1$, then

$$\int_0^1 H_1(u; T_m) du \leq T_1^{\max} \leq \rho_1 T_m.$$

Since $T_m = T_{\text{OPT}}$, the theorem follows. $\square$

C. Asymptotic Randomized Lower-Bound

Lemma 13 gives a randomized online threshold algorithm, namely the static helper DSR, with expected competitive ratio ρ_1 . We now show that this improvement cannot be asymptotically surpassed by any randomized online algorithm. Informally, the lower bound is proved by relating randomized offloading decisions to a deterministic fractional decision problem: a randomized algorithm induces on every instance a probability distribution over offloading stages, which can be viewed as a fractional allocation across those stages. Thus, any lower bound for the corresponding fractional problem also lower bounds the expected performance of randomized online offloading.

Let $\mathcal{A}_{\text{det}}$ be the set of deterministic algorithms for Online Delay Minimization Problem over instance set $\mathcal{I}$. Each $a \in \mathcal{A}_{\text{det}}$ maps an instance to an offloading stage:

$$a : \mathcal{I} \rightarrow \mathcal{L} \cup \{L+1\}.$$

A randomized online algorithm ALG_p for Online Delay Minimization Problem is specified by a probability distribution over $\mathcal{A}_{\text{det}}$:

$$p : \mathcal{A}_{\text{det}} \rightarrow [0, 1], \quad \int_{\mathcal{A}_{\text{det}}} dp(a) = 1.$$

Given $I \in \mathcal{I}$, ALG_p samples $a \sim p$ and executes a . Its expected total delay is

$$T_{\text{ALG}_p(I)} := E_{a \sim p}[T_a(I)] = \int_{\mathcal{A}_{\text{det}}} T_a(I) dp(a). \quad (16)$$

Define a hypothetical online problem P' over $\mathcal{I}$ with an objective to find a deterministic algorithm $\text{ALG}' : \mathcal{I} \rightarrow \Delta^{L+1}$ that minimizes the cost

$$T_{\text{ALG}'(I)} = \sum_{i=1}^{L+1} s_i T_i \quad (17)$$

where $\Delta^{L+1} = \{(s_1, \dots, s_{L+1}) \in \mathbb{R}_{\geq 0}^{L+1} \mid \sum_{i=1}^{L+1} s_i = 1\}$.

Lemma 17. *Given any randomized algorithm ALG_p for online delay minimization problem, there exists a deterministic*

algorithm ALG' for P' such that for any delay sequence I , $T_{\text{ALG}_p(I)} = T_{\text{ALG}'(I)}$.

Proof. From (16), the expected makespan of ALG_p is $\int_{\mathcal{A}_{\text{det}}} T_{a(I)} dp(a)$. Consider a deterministic algorithm ALG' with $s_i = \int_{\mathcal{A}_{\text{det}}} \mathbb{I}\{a(I) = i\} dp(a)$. Then, from (17) the cost of ALG' is

$$\begin{aligned} T_{\text{ALG}'(I)} &= \sum_{i=1}^{L+1} \left(\int_{\mathcal{A}_{\text{det}}} \mathbb{I}\{a(I) = i\} dp(a) \right) T_i \\ &= \int_{\mathcal{A}_{\text{det}}} \sum_{i=1}^{L+1} T_i \mathbb{I}\{a(I) = i\} dp(a) \\ &= \int_{\mathcal{A}_{\text{det}}} T_{a(I)} dp(a) = E_{a \sim p}[T_{a(I)}]. \end{aligned}$$

□

Let $A = T^{\max} - T^{\min}$. Define ρ as the solution to the following equation:

$$\rho = \frac{1}{(L+1)(1-q)}, \quad (18)$$

where

$$q = \left(\frac{(L+1)(\rho-1)T^{\max}}{(\rho(L+1)-1)A} \right)^{1/L}. \quad (19)$$

Consider an instance $\hat{I}$ defined as $\hat{P}_k^r = \hat{P}_k^l = 0, \forall k < L, \hat{P}_L^r > 0, \hat{P}_L^l = T^{\min} = \alpha_{\min} P_L^r$ and $\hat{C}_k = T^{\max} - T^r - Aq^{L+1-k} \forall k \in \{1, \dots, L\}$ where $T^{\max} = \alpha_{\max} P_L^r$ and $T^r = P_L^r$. Then by plugging these processing and communication delays into (2) we can obtain the costs

$$\hat{T}_k = T^{\max} - Aq^{L+1-k} \forall k \in \{1, \dots, L+1\}. \quad (20)$$

From the definition of $\hat{T}_k$ it is easy to see that $\hat{T}_{k+1} \leq \hat{T}_k$ for $k \in \{1, \dots, L\}$.

We now give the following result for this special instance $\hat{I}$.

Lemma 18. For every $i \in \{1, \dots, L+1\}$,

$$\frac{1}{L+1} \sum_{k=1}^i \hat{T}_k + \frac{L+1-i}{L+1} T^{\max} = \rho \hat{T}_i.$$

Proof. Let

$$D_i = \frac{1}{L+1} \sum_{k=1}^i \hat{T}_k + \frac{L+1-i}{L+1} T^{\max}.$$

Then D_i can be simplified as

$$\begin{aligned} D_i &\stackrel{(a)}{=} \frac{i}{L+1} T^{\max} - \frac{1}{L+1} \sum_{k=1}^i Aq^{L+1-k} + \frac{L+1-i}{L+1} T^{\max} \\ &= T^{\max} - \frac{1}{L+1} \left(Aq^{L+1-i} \frac{1-q^i}{1-q} \right) \\ &\stackrel{(b)}{=} T^{\max} - \rho Aq^{L+1-i} (1-q^i) \\ &= T^{\max} - \rho A (q^{L+1-i} - q^{L+1}) \end{aligned}$$

$$\begin{aligned} &= \rho \hat{T}_i + (1-\rho) T^{\max} + \rho Aq^{L+1} \\ &\stackrel{(c)}{=} \rho \hat{T}_i + (1-\rho) T^{\max} \\ &+ \left(\rho - \frac{1}{L+1} \right) A \left(\frac{(L+1)(\rho-1)T^{\max}}{(\rho(L+1)-1)A} \right) \\ &= \rho \hat{T}_i \end{aligned} \quad (21)$$

Here the equality (a) is from the definition of $\hat{T}_k$. The equality (b) is from equation (18). Equality (c) is obtained by replacing q^L using equation (19) and ρq obtained from equation (18). □

Next we give the following results for $\{s_k\}_{k=1}^{L+1}$ with special properties that lower bound linear functions of $\{s_k\}_{k=1}^{L+1}$.

Lemma 19. Given a $u \in \{1, \dots, L+1\}$, for any $\{s_k\}_{k=1}^{L+1}$ that satisfies,

$$\sum_{k=1}^i s_k \geq \frac{i}{L+1} \forall i \in \{1, \dots, u\},$$

we have

$$\sum_{k=1}^u s_k \hat{T}_k \geq \frac{1}{L+1} \sum_{k=1}^u \hat{T}_k$$

where it holds with equality when $s_k = 1/(L+1)$ for all $k \in \{1, \dots, L+1\}$.

Proof. We prove this by contradiction. Let

$$\sum_{k=1}^u s_k \hat{T}_k < \frac{1}{L+1} \sum_{k=1}^u \hat{T}_k \quad (22)$$

for some $\{s_k\}_{k=1}^{L+1}$ such that

$$\sum_{k=1}^i s_k \geq \frac{i}{L+1} \forall i \in \{1, \dots, u\}. \quad (23)$$

Since $\hat{T}_k$ is strictly decreasing in k , the inequality in (22) can hold only if the weight vector $(s_1, \dots, s_u)$ places more mass on larger indices than the uniform vector $(1/(L+1), \dots, 1/(L+1))$. In particular, there must exist $l < m \leq u$ such that

$$s_l < \frac{1}{L+1} \quad \text{and} \quad s_m > \frac{1}{L+1}.$$

Equivalently, there exists $v < u$ such that

$$\sum_{k=v+1}^u s_k > \sum_{k=v+1}^u \frac{1}{L+1} = \frac{u-v}{L+1}. \quad (24)$$

But if (24) holds, then we have

$$\sum_{k=1}^v s_k = \frac{u}{L+1} - \sum_{k=v}^u s_k \stackrel{(a)}{<} \frac{v}{L+1}$$

which contradicts (23). Here (a) is from (24). This proves the lemma. □

Proof. Let

$$\delta_k = s_k - \frac{1}{L+1}, \quad A_i = \sum_{k=1}^i \delta_k, \quad i \in \{1, \dots, u\}.$$

From the constraints on $\{s_k\}_{k=1}^{L+1}$,

$$A_i = \sum_{k=1}^i s_k - \frac{i}{L+1} \geq 0, \quad \forall i \in \{1, \dots, u\}.$$

Note that $\delta_i = A_i - A_{i-1}$. By definition, $\hat{T}_k$ is decreasing in k . Hence, by summation by parts,

$$\begin{aligned} \sum_{k=1}^u \left(s_k - \frac{1}{L+1} \right) \hat{T}_k &= \sum_{k=1}^u \delta_k \hat{T}_k \\ &= \sum_{k=1}^u A_k \hat{T}_k - \sum_{k=1}^u A_{k-1} \hat{T}_k \\ &= A_u \hat{T}_u + \sum_{k=1}^{u-1} A_k (\hat{T}_k - \hat{T}_{k+1}). \end{aligned}$$

Since $A_k \geq 0$ for all $k \leq u$ and $\hat{T}_k - \hat{T}_{k+1} \geq 0$, the right-hand side is nonnegative. Therefore,

$$\sum_{k=1}^u \left(s_k - \frac{1}{L+1} \right) \hat{T}_k \geq 0,$$

which implies

$$\sum_{k=1}^u s_k \hat{T}_k \geq \frac{1}{L+1} \sum_{k=1}^u \hat{T}_k.$$

Lemma 20. Given a $u \in \{1, \dots, L\}$, for any $\{s_k\}_{k=1}^{L+1}$ that satisfies,

$$\sum_{k=1}^i s_k \geq \frac{i}{L+1} \quad \forall i \in \{1, \dots, u\} \quad (25)$$

and

$$\sum_{k=1}^{u+1} s_k < \frac{u+1}{L+1}, \quad (26)$$

we have

$$\sum_{k=1}^{u+1} s_k \hat{T}_k + \sum_{k=u+2}^{L+1} s_k T^{\max} > \rho \hat{T}_{u+1}.$$

Proof. Taking a compliment of (26) we get

$$\sum_{k=u+2}^{L+1} s_k > \frac{L-u}{L+1}$$

Define a slack variable

$$\delta = \sum_{k=u+2}^{L+1} s_k - \frac{L-u}{L+1} > 0. \quad (27)$$

Then we have

$$\sum_{k=1}^{u+1} s_k = 1 - \sum_{k=u+2}^{L+1} s_k = \frac{u+1}{L+1} - \delta.$$

Upon rearranging the above, we get

$$s_{u+1} = \frac{u+1}{L+1} - \sum_{k=1}^u s_k - \delta \quad (28)$$

Then

$$\begin{aligned} \sum_{k=1}^{u+1} s_k \hat{T}_k + \sum_{k=u+2}^{L+1} s_k T^{\max} &\stackrel{(a)}{=} \sum_{k=1}^{u+1} s_k \hat{T}_k + \frac{L-u}{L+1} T^{\max} + \delta T^{\max} \\ &\stackrel{(b)}{\geq} \frac{1}{L+1} \sum_{k=1}^u \hat{T}_k + s_{u+1} \hat{T}_{u+1} \\ &\quad + \frac{L-u}{L+1} T^{\max} + \delta T^{\max} \\ &\stackrel{(c)}{=} \frac{1}{L+1} \sum_{k=1}^{u+1} \hat{T}_k - \delta \hat{T}_{u+1} \\ &\quad + \frac{L-u}{L+1} T^{\max} + \delta T^{\max} \\ &\stackrel{(d)}{>} \frac{1}{L+1} \sum_{k=1}^{u+1} \hat{T}_k + \frac{L-u}{L+1} T^{\max} \\ &\stackrel{(e)}{=} \rho T_{u+1} \end{aligned}$$

The inequality (a) follows from (27). Inequality (b) is from Lemma 19. Inequality (c) is from (28) where $\sum_{k=1}^u s_k = u/(L+1)$. Inequality (d) is from the fact that $T^{\max} > T_{u+1}$. Equality (e) is from Lemma 18. $\square$

Lemma 21. If

$$\sum_{k=1}^{L+1} s_k \hat{T}_k \geq \rho \hat{T}_{L+1} = \rho T^{\min}. \quad (29)$$

then

$$\sum_{k=1}^i s_k \geq \frac{i}{L+1} \quad \forall i \in \{1, \dots, L+1\}.$$

Proof. From Lemma 19 for $u = L+1$ we have

$$\sum_{k=1}^{L+1} s_k \hat{T}_k \geq \frac{1}{L+1} \sum_{k=1}^{L+1} \hat{T}_k.$$

Then, from Lemma 18 for $i = L+1$,

$$\frac{1}{L+1} \sum_{k=1}^{L+1} \hat{T}_k = \rho \hat{T}_{L+1}.$$

The proof follows from the fact that $\hat{T}_{L+1} = T^{\min}$ from the definition of $\hat{T}_k$. $\square$

Now we provide the lower bound for P' .

Lemma 22. *No algorithm for P' can achieve a competitive ratio smaller than ρ .*

Proof. Let ALG' be any deterministic algorithm for P' . Using an adversarial argument we show that ALG' cannot achieve a ratio smaller than ρ . Let $\text{ALG}'(\hat{I}) = \{\hat{s}_k\}_{k=1}^{L+1}$. Then we define a set

$$J = \left\{ i \in \mathcal{L} \cup \{L+1\} \mid \sum_{k=1}^i \hat{s}_k < \frac{i}{L+1} \right\}$$

- *Case 1:* If $J = \emptyset$, then

$$\sum_{k=1}^i \hat{s}_k \geq \frac{i}{L+1} \quad \forall i \in \{1, \dots, L\}.$$

For such $\{\hat{s}_k\}_{k=1}^{L+1}$, from Lemma 21 we know that ALG' cannot achieve a ratio smaller than ρ .

- *Case 2:* If $J \neq \emptyset$, then define $j = \min J$. The adversary then sets the future costs $\hat{C}_k = T^{\max} - T^r$ for $k \in \{j+1, \dots, L\}$ and $\hat{P}_L = \alpha_{\max} \hat{P}_L^r$. Then from (2) we have $\hat{T}_i = T^{\max} \quad \forall i \in \{j+1, L+1\}$. Let this modified instance be $\hat{I}_j^{\max}$. Since $\hat{T}_k$ is decreasing in k , $\text{OPT}(\hat{I}_j^{\max}) = \hat{T}_j$. Therefore, in this case as well on setting $u = j-1$ in Lemma 20 it can be seen that ALG' cannot achieve a competitive ratio smaller than ρ . $\square$

Theorem 23. *No randomized algorithm can achieve a competitive ratio smaller than ρ for online delay minimization problem.*

Proof. From Lemma 17 $T_{\text{ALG}_p(I)} = T_{\text{ALG}'(I)}$. Since $\text{OPT}(I) = \text{OPT}'(I)$, we have $\rho_{\text{ALG}_p} = \rho_{\text{ALG}'}$. From Lemma 22 we have $\rho_{\text{ALG}'} \geq \rho$. $\square$

Corollary 24. *For large L , no randomized algorithm can do better than*

$$\lim_{L \rightarrow \infty} \rho = \frac{1}{1 + W\left(\frac{1 - \alpha_{\max}/\alpha_{\min}}{e\alpha_{\max}/\alpha_{\min}}\right)}.$$

Proof. Using equation (19), q^{L+1} can be written as

$$\begin{aligned} q^{L+1} &= q \left(\frac{(L+1)(\rho-1)T^{\max}}{(\rho(L+1)-1)(T^{\max}-T^{\min})} \right) \\ &= \frac{\rho(L+1)-1}{\rho(L+1)} \left(\frac{(L+1)(\rho-1)T^{\max}}{(\rho(L+1)-1)(T^{\max}-T^{\min})} \right) \\ &= \left(\frac{1-1/\rho}{1-T^{\min}/T^{\max}} \right) \\ &= \left(\frac{1-1/\rho}{1-\alpha_{\min}/\alpha_{\max}} \right). \end{aligned}$$

Then by replacing the q in equation (18) by the above, we get

$$\left(\frac{1-1/\rho}{1-\alpha_{\min}/\alpha_{\max}} \right) = \left(1 - \frac{1}{(L+1)\rho} \right)^{L+1}.$$

When $L \rightarrow \infty$, we have

$$\left(\frac{1-1/\rho}{1-\alpha_{\min}/\alpha_{\max}} \right) = e^{-1/\rho}$$

After rearranging, this can be rewritten as

$$\left(\frac{1}{\rho} - 1 \right) e^{\frac{1}{\rho}-1} = -\frac{1}{e} \left(1 - \frac{\alpha_{\min}}{\alpha_{\max}} \right).$$

The proof follows by plugging the definition of Lambert W : if $ze^z = a$ then $z = W(a)$. $\square$

VII. NUMERICAL EVALUATION

We study the numerical performance of DOPart, with experiments on AlexNet, VGG16, and ResNet34. The stages of AlexNet and VGG16 directly correspond to their layers of neurons, while the stages of ResNet34 are more complex as shown in Fig. 1.

The remote processing latencies P_k^r for each stage are obtained by executing 5000 samples each of AlexNet, VGG19, and ResNet34 inference jobs on a dedicated Digital Research Alliance of Canada cluster with Nvidia-T4 GPUs. The local processing latency for stage k is set to $P_k^l = 2^\mu P_k^r$ where μ is drawn from the uniform distribution between $\log_2 \alpha_{\min}$ and $\log_2 \alpha_{\max}$, to emulate the fluctuation in the local machine's processing speed due to random demands from parallel applications on the same machine. We simulate a random communication channel for offloading, where the bandwidth B_i for stage i is drawn from the uniform distribution between 5Mbps and 60Mbps, which emulates the wireless fading channel of a mobile device.

We compare DOPart against the following benchmarks:

- **Neuro:** Neurosurgeon [4] predicts the future processing and communication delays based on a pre-trained model. We implement the best version of Neurosurgeon by always using the value that minimizes the prediction error for the future processing latencies of all the layers. It uses only the network bandwidth at the beginning of the execution of the inference job. We remark that the method in [10] behaves the same way as Neurosurgeon for sequentially dependent tasks, so this benchmark also represents [10].
- **AutoNeuro:** Autodidactic Neurosurgeon [8] uses an on-line learning approach to solve the sequential dependent task partitioning problem. To compare with [8] in its most favourable performance, we implement the oracle from [8], which is the idealized best prediction that one can achieve. For each experimental setting we do so by finding the partition point that minimizes the average delay over 5000 executions of the inference job.
- **Local Only:** This is a trivial strategy that execute the inference job locally for T^l total delay.
- **Remote Only:** This is a trivial strategy that offloads the complete inference job to the remote device, which incurs $C_1 + T^r$ total delay.

In Fig. 2, we present the average delay of DOPart and all benchmarks over 5000 instances of inference jobs on ResNet34. We observe that DOPart can substantially outperform both Neuro and AutoNeuro. Consider an example of a typical scenario where $\alpha_{\min} = 1$ and $\alpha_{\max} = 4$. DOPart provides 17% improvement over Neuro and 13% improvement

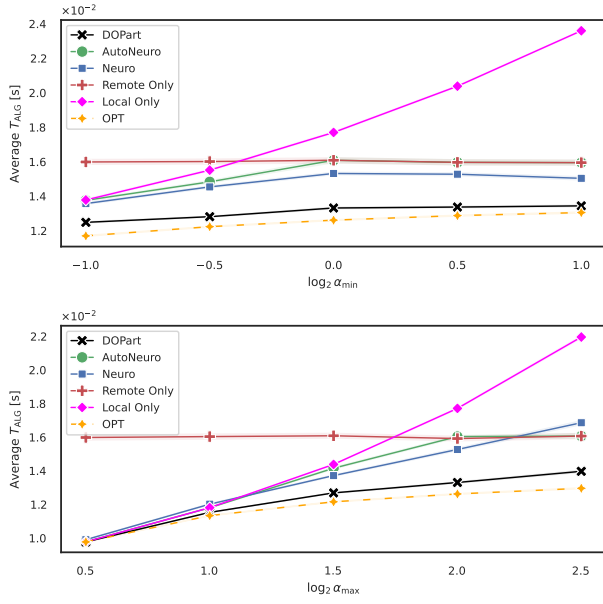


Fig. 2. Average delay for ResNet34, with 95% confidence intervals.

over AutoNeuro. Furthermore, we observe that the average delay increases with $\alpha_{\min}$ and $\alpha_{\max}$. This is because the expected local processing latencies increase with increasing $\alpha_{\min}$ and $\alpha_{\max}$. Similar observation are made with AlexNet and VGG19. We omit those figures to avoid redundancy.

We are also interested in the impact of the estimation accuracy of $\alpha_{\min}$ and $\alpha_{\max}$ on the performance of DOPart. Fig. 3 shows how the average empirical performance ratio between of DOPart and the optimal offline algorithm responds to changes in the estimation errors ϵ_1 and ϵ_2 on $\alpha_{\min}$ and $\alpha_{\max}$, respectively. We set $\alpha_{\min} = 1$ and $\alpha_{\max} = 3$. Our main observation is that DOPart still achieves excellent performance ratios even for up to 20% ~ 30% estimation error. Furthermore, it is interesting to see that for AlexNet and ResNet34 the performance initially improves, but for VGG19 it degrades, with underestimation of both $\alpha_{\min}$ and $\alpha_{\max}$. The results are reversed for overestimation of $\alpha_{\min}$ and $\alpha_{\max}$. This suggests that there is no general trend on whether one should favour underestimation or overestimation.

VIII. CONCLUSION

In this paper, we propose a lightweight online Delay-optimal Online Partitioning (DOPart) algorithm for the offloading of computation jobs with sequentially dependent tasks. Through dynamic adaptation to the unpredictable fluctuation in the system state, it encourages optimal sharing between local and remote computational resources while considering the impact of offloading delay. We obtain the competitive ratio of DOPart and show that it is optimal, by providing a matching lower bound for the delay minimization problem. We further observe that DOPart is robust against estimation errors on $\alpha_{\min}$ and $\alpha_{\max}$, in terms of both the competitive ratio and numerical average performance.

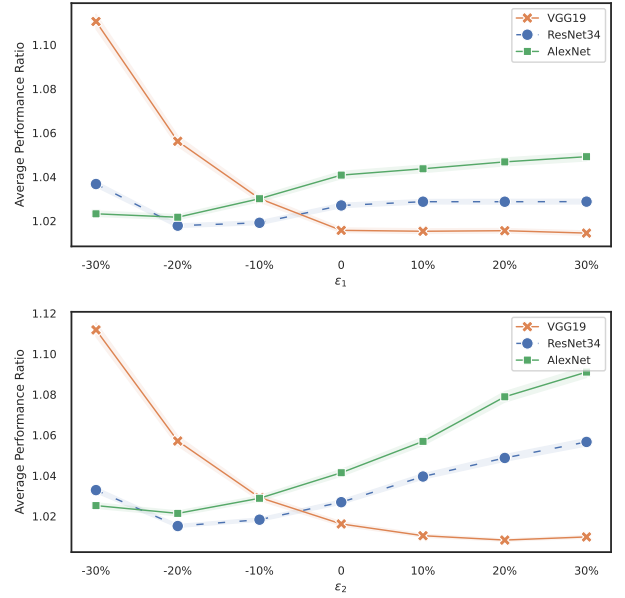


Fig. 3. Performance ratio vs. parameter estimation errors.

APPENDIX A MULTIPLE DEVICES

Let M be the number of devices in the system. Each device has an inference Job that has to be processed. The local jobs are processed parallelly and the remote jobs are on a shared server. Let $s_m \in \{1, \dots, L+1\}$ denote the offloading stage selected for device m , where $s_m = L+1$ means that device m is processed entirely locally. For $s_m \leq L$, define the release time and remote processing time as

$$R_m(s_m) = \sum_{k=1}^{s_m-1} P_{k,m}^l + C_{s_m,m},$$

and

$$D_m(s_m) = \sum_{k=s_m}^L P_{k,m}^r.$$

Define remote start and finish time of under schedule π as $S_{\pi_m} = \max\{R_{\pi_j}(s_{\pi_j}), F_{\pi_{j-1}}\}$, $F_{\pi_j} = S_{\pi_j} + D_{\pi_j}(s_{\pi_j})$, respectively.

For local device

$$F_m = T_m^l = \sum_{k=1}^L P_{k,m}^l, \quad \text{if } s_m = L+1.$$

The makespan objective

$$\min_{\{s_m\}, \pi} T_{\max}$$

subject to

$$T_{\max} \geq F_m, \quad \forall m.$$

$$\min_{\{s_m\}, \pi} \max_m F_m.$$

Let $\mathcal{M} = \{1, \dots, M\}$ denote the set of devices. Each device $m \in \mathcal{M}$ has one inference job. Local computation is performed in parallel across devices, while all offloaded computation is processed by a shared remote server.

Let $\mathcal{Q}(t)$ denote the set of offloaded jobs that have arrived at the remote server by time t and have not yet completed. The server workload, or weight of the jobs currently remaining in the system, is defined as

$$W(t) = \sum_{m \in \mathcal{Q}(t)} d_m(t),$$

where $d_m(t)$ is the remaining remote processing time of job m at time t .

$$\mathcal{Q}(t) = \{m \in \mathcal{M} : s_m \leq L, R_m(s_m) \leq t < F_m\}.$$

$$W(t) = \sum_{m \in \mathcal{Q}(t)} d_m(t).$$

$$d_m(t) = \begin{cases} D_m(s_m), & R_m(s_m) \leq t < S_m, \\ F_m - t, & S_m \leq t < F_m, \\ 0, & t \geq F_m, \end{cases}$$

The weight of the jobs currently remaining on the remote server at time t , also called the server workload, is the total unfinished remote processing time:

$$W(t) = \sum_{m \in \mathcal{Q}(t)} d_m(t),$$

where

$$\mathcal{Q}(t) = \{m \in \mathcal{M} : s_m \leq L, R_m(s_m) \leq t < F_m\}$$

is the set of offloaded jobs that have arrived at the server and have not yet completed, and $d_m(t)$ is the remaining remote processing time of job m .

Let $\mathcal{L}(\mathbf{s})$ denote the set of devices whose jobs are executed entirely locally:

$$\mathcal{L}(\mathbf{s}) = \{m \in \mathcal{M} : s_m = L + 1\}.$$

For each $m \in \mathcal{L}(\mathbf{s})$, its completion time is

$$T_m^l = \sum_{k=1}^L P_{k,m}^l.$$

$$\hat{t} = \min\{t \geq 0 : W(t) = 0\},$$

$$\hat{t} = \max_{m \in \mathcal{M} \setminus \mathcal{L}(\mathbf{s})} F_m,$$

$$\min_{\mathbf{s}, \pi} \max \left\{ \hat{t}, \max_{m \in \mathcal{L}(\mathbf{s})} T_m^l \right\}.$$

The overall makespan is the maximum of two quantities: the time $\hat{t}$ at which the shared remote server finishes all offloaded jobs, and the latest completion time among the

jobs executed entirely locally. Therefore, the multiple-device offloading problem is

$$\min_{\mathbf{s}, \pi} \max \left\{ \hat{t}, \max_{m \in \mathcal{L}(\mathbf{s})} \sum_{k=1}^L P_{k,m}^l \right\}.$$

$$R_{m,i} = \sum_{k=1}^{i-1} P_{k,m}^l + C_{i,m}, \quad i = 1, \dots, L,$$

$$D_{m,i} = \sum_{k=i}^L P_{k,m}^r, \quad i = 1, \dots, L,$$

and

$$T_m^l = \sum_{k=1}^L P_{k,m}^l.$$

$$x_{m,i} = \begin{cases} 1, & \text{if device } m \text{ offloads at stage } i, \\ 0, & \text{otherwise,} \end{cases} \quad i = 1, \dots, L+1.$$

$$y_m = \sum_{i=1}^L x_{m,i}$$

$$z_{m,n} = \begin{cases} 1, & \text{if offloaded job } m \text{ is processed before offloaded job } n, \\ 0, & \text{otherwise,} \end{cases}$$

$$\min T_{\max}$$

$$\text{s.t.} \quad \sum_{i=1}^{L+1} x_{m,i} = 1, \quad \forall m \in \mathcal{M},$$

$$y_m = \sum_{i=1}^L x_{m,i}, \quad \forall m \in \mathcal{M},$$

$$S_m \geq \sum_{i=1}^L R_{m,i} x_{m,i} - H(1 - y_m), \quad \forall m \in \mathcal{M},$$

$$F_m \geq S_m + \sum_{i=1}^L D_{m,i} x_{m,i} - H(1 - y_m), \quad \forall m \in \mathcal{M},$$

$$S_n \geq F_m - H(1 - z_{m,n}) - H(2 - y_m - y_n), \quad \forall m < n,$$

$$S_m \geq F_n - H z_{m,n} - H(2 - y_m - y_n), \quad \forall m < n,$$

$$T_{\max} \geq F_m - H(1 - y_m), \quad \forall m \in \mathcal{M},$$

$$T_{\max} \geq T_m^l x_{m,L+1}, \quad \forall m \in \mathcal{M},$$

$$x_{m,i} \in \{0, 1\}, \quad \forall m \in \mathcal{M}, i = 1, \dots, L+1,$$

$$y_m \in \{0, 1\}, \quad \forall m \in \mathcal{M},$$

$$z_{m,n} \in \{0, 1\}, \quad \forall m < n,$$

$$S_m, F_m, T_{\max} \geq 0, \quad \forall m \in \mathcal{M}.$$

$$H = \sum_{m \in \mathcal{M}} \max_{i=1, \dots, L} D_{m,i} + \max_{m \in \mathcal{M}, i=1, \dots, L} R_{m,i} + \max_{m \in \mathcal{M}} T_m^l.$$

$$\min \max \left\{ \hat{t}, \max_{m \in \mathcal{L}(s)} T_m^l \right\},$$

- What does the device m know at the stage i about the server and other devices M/m
- It does not know if the device m' is offloading at the stage j for $C_{j,m'}$
- At time $t = 0$, we know that each device has T^r work remaining.
- So if each device sees the communication delay $C_{1,m}$
- Solve local problems of minimizing the maximum between projected local delay and projected server delay.

For a candidate makespan T , the central controller maintains the current server clearing time

$$A(t) = t + W(t),$$

where $W(t)$ is the unfinished remote workload at time t . When device m reaches stage k , the controller computes the worst-case remaining local completion time

$$F_{m,k}^{\text{loc}}(t) = t + \alpha_{\max} \sum_{j=k}^L P_{j,m}^r,$$

and the completion time if the job is offloaded immediately,

$$F_{m,k}^{\text{off}}(t) = \max\{t + C_{k,m}, A(t)\} + \sum_{j=k}^L P_{j,m}^r.$$

If $F_{m,k}^{\text{loc}}(t) > T$ and $F_{m,k}^{\text{off}}(t) \leq T$, then the controller offloads device m at stage k . If $F_{m,k}^{\text{loc}}(t) \leq T$, the job may safely continue locally. If neither local continuation nor immediate offloading can meet T , the target makespan is increased.

If a target T is feasible under the worst-case local delays $P_{k,m}^l = \alpha_{\max} P_{k,m}^r$, then the algorithm never misses a local completion that could violate T . Whenever local execution is no longer guaranteed to finish by T , the controller either offloads the job while the server can still complete it by T , or certifies that the current target is too small and increases T .

Since the optimal makespan is not known in advance, the controller uses a doubling strategy. It maintains a target T , initialized to a lower bound on the optimal makespan. Whenever the current job cannot be guaranteed to complete by T either by local continuation or by immediate offloading, the controller updates $T \leftarrow 2T$. Thus, after the final update, the maintained target is within a factor of two of the smallest feasible target.

Lemma 25. *If the target T is at least the robust offline optimum T_{rob}^* , then the target- T algorithm completes all jobs by time T .*

Algorithm 3: Target-Makespan Online Offloading

Input: Initial target T , bounds $\alpha_{\min}, \alpha_{\max}$

- 1 Maintain the server clearing time $A(t) = t + W(t)$;
- 2 **for** each decision epoch t , when device m reaches stage k **do**
- 3 Compute

$$F_{m,k}^{\text{loc}}(t) = t + \alpha_{\max} \sum_{j=k}^L P_{j,m}^r.$$
 Compute

$$F_{m,k}^{\text{off}}(t) = \max\{t + C_{k,m}, A(t)\} + \sum_{j=k}^L P_{j,m}^r.$$
while $F_{m,k}^{\text{loc}}(t) > T$ and $F_{m,k}^{\text{off}}(t) > T$ **do**
 | $T \leftarrow 2T$;
- 5 **end**
- 6 **if** $F_{m,k}^{\text{loc}}(t) > T$ **then**
- 7 Offload device m at stage k ;
- 8 Update

$$A(t) \leftarrow F_{m,k}^{\text{off}}(t).$$
- 9 **end**
- 10 **else**
- 11 Continue processing device m locally;
- 12 **end**
- 13 **end**

Theorem 26. *Assume $C_{k,m} = 0$ for all k, m , and*

$$\alpha_{\min} P_k^r \leq P_{k,m}^l \leq \alpha_{\max} P_k^r.$$

If the target-makespan algorithm uses doubling and its target- T subroutine completes every robustly feasible instance by time T , then its competitive ratio is at most

$$2 \frac{\alpha_{\max}}{\alpha_{\min}}.$$

Let T^* be the offline optimal makespan for the realized instance, and let T_{rob}^* be the offline optimal makespan under worst-case local delays $P_{k,m}^l = \alpha_{\max} P_k^r$. Since every realized local delay is at least $\alpha_{\min} P_k^r$, scaling any optimal realized schedule by $\alpha_{\max}/\alpha_{\min}$ gives a feasible robust schedule. Hence

$$T_{\text{rob}}^* \leq \frac{\alpha_{\max}}{\alpha_{\min}} T^*.$$

By the doubling rule, after the final target update,

$$T_f < 2T_{\text{rob}}^*.$$

Since the target- T subroutine completes all jobs by time T_f ,

$$T_{\text{ALG}} \leq T_f < 2T_{\text{rob}}^* \leq 2 \frac{\alpha_{\max}}{\alpha_{\min}} T^*.$$

Therefore the competitive ratio is at most

$$2\alpha_{\max}/\alpha_{\min}.$$

Algorithm 4: Doubling Target-Makespan Offloading

Input: Initial target T_0 , bounds $\alpha_{\min}, \alpha_{\max}$

- 1 Set $T \leftarrow T_0$;
- 2 Maintain the server clearing time $A(t) = t + W(t)$;
- 3 **for** each decision epoch t , when device m reaches stage k **do**
- 4 Compute

$$F_{m,k}^{\text{loc}}(t) = t + \alpha_{\max} \sum_{j=k}^L P_{j,m}^r.$$

 Compute

$$F_{m,k}^{\text{off}}(t) = \max\{t + C_{k,m}, A(t)\} + \sum_{j=k}^L P_{j,m}^r.$$
- 5 **while** $F_{m,k}^{\text{loc}}(t) > T$ and $F_{m,k}^{\text{off}}(t) > T$ **do**
- 6 $T \leftarrow 2T$;
- 7 **end**
- 8 **if** $F_{m,k}^{\text{loc}}(t) \leq T$ **then**
- 9 Continue processing device m locally;
- 10 **end**
- 11 **else**
- 12 Offload device m at stage k ;
- 13 Update

$$A(t) \leftarrow F_{m,k}^{\text{off}}(t).$$
- 14 **end**
- 15 **end**

When $C_{k,m} = 0$ and $P_{k,m}^l = \alpha P_k^r$ for a known constant α , the multiple-device problem reduces to balancing the local prefix time against the total work assigned to the shared remote server. Let $D_s = \sum_{k=s}^L P_k^r$ and $D_{L+1} = 0$. If device m offloads at stage s_m , its local prefix time is $\alpha(T^r - D_{s_m})$, while it contributes D_{s_m} units of work to the remote server. Hence the optimal makespan is

$$T^* = \min_{s_1, \dots, s_M \in \{1, \dots, L+1\}} \max \left\{ \max_m \alpha(T^r - D_{s_m}), \sum_{m=1}^M D_{s_m} \right\}.$$

Under the restriction $s_m \in \{1, L+1\}$, each device is either offloaded entirely at the beginning or processed entirely locally. With zero communication delay, if $\mathcal{O}$ is the set of offloaded devices, the makespan is

$$T(\mathcal{O}) = \max \left\{ |\mathcal{O}|T^r, \max_{m \notin \mathcal{O}} T_m^l \right\}.$$

Hence the offline optimum is

$$T^* = \min_{\mathcal{O} \subseteq \mathcal{M}} \max \left\{ |\mathcal{O}|T^r, \max_{m \notin \mathcal{O}} T_m^l \right\}.$$

Equivalently, after sorting $T_{(1)}^l \geq \dots \geq T_{(M)}^l$,

$$T^* = \min_{n=0, \dots, M} \max\{nT^r, T_{(n+1)}^l\},$$

where $T_{(M+1)}^l = 0$.

Algorithm 5: Doubling Target-Makespan Offloading

Input: Initial target T_0 , bounds $\alpha_{\min}, \alpha_{\max}$

- 1 Set $T \leftarrow T_0$;
- 2 Define $D_k = \sum_{j=k}^L P_j^r$, for $k = 1, \dots, L$;
- 3 Maintain the server clearing time $A(t) = t + W(t)$;
- 4 **for** each decision epoch t , when device m reaches stage k **do**
- 5 Set

$$F_k^{\text{loc}}(t) = t + \alpha_{\max} D_k.$$

 Set

$$F_{m,k}^{\text{off}}(t) = \max\{t + C_{k,m}, A(t)\} + D_k.$$
- 6 **while** $F_k^{\text{loc}}(t) > T$ and $F_{m,k}^{\text{off}}(t) > T$ **do**
- 7 $T \leftarrow 2T$;
- 8 **end**
- 9 **if** $F_k^{\text{loc}}(t) \leq T$ **then**
- 10 Continue processing device m locally;
- 11 **end**
- 12 **else**
- 13 Offload device m at stage k ;
- 14 Update

$$A(t) \leftarrow F_{m,k}^{\text{off}}(t).$$
- 15 **end**
- 16 **end**

APPENDIX B MULTIPLE JOBS

There exist N identical jobs in the system that have to be processed. Let $T_1 = C_1 + T^r$ be the cost of offloading a job at stage 1 and $T_{L+1} = T^l$ be the cost of processing the entire job locally.

The local jobs are processed parallelly and the remote jobs are provided dedicated computation resources (e.g. VMs).

At the first stage, the ALG offloads N_1^{ALG} jobs for T_1 . Let $\eta_1^{\text{ALG}} = N_1^{\text{ALG}}/N$. Similarly, for OPT define N_1^{OPT} and η_1^{OPT} .

A. Special Instances

Consider the instances where $C_i \gg 0 \ \forall \ i > 1$. The makespan of completing N jobs for an arbitrary η_1 is given as

$$M = \max\{\eta_1(C_1 + T^r), (1 - \eta_1)T^l\}$$

Lets consider two cases:

- $T^l = \alpha_{\min} T^r$: In this case

$$\eta_1^{\min} = \frac{\alpha_{\min} T^r}{\alpha_{\min} T^r + C_1 + T^r}$$

and

$$M^{\min} = \frac{\alpha_{\min} T^r (C_1 + T^r)}{\alpha_{\min} T^r + C_1 + T^r}$$

- $T^l = \alpha_{\max} T^r$: In this case

$$\eta_1^{\max} = \frac{\alpha_{\max} T^r}{\alpha_{\max} T^r + C_1 + T^r}$$

and

$$M^{\max} = \frac{\alpha_{\max} T^r (C_1 + T^r)}{\alpha_{\max} T^r + C_1 + T^r}$$

Let the algorithm ALG offload η_1^{ALG} fraction of jobs at stage 1. Then the makespan of ALG is given as

$$M^{\text{ALG}} = \max\{\eta_1^{\text{ALG}}(C_1 + T^r), (1 - \eta_1^{\text{ALG}})T^l\}$$

When T^l is:

- $\alpha_{\min} T^r$: $M^{\text{ALG}} = \eta_1^{\text{ALG}}(C_1 + T^r)$
- $\alpha_{\max} T^r$: $M^{\text{ALG}} = (1 - \eta_1^{\text{ALG}})\alpha_{\max} T^r$

Then the competitive ratio when T^l is:

- $\alpha_{\min} T^r$:

$$\rho(\eta_1^{\text{ALG}}, C_1) = \frac{\eta_1^{\text{ALG}}(C_1 + T^r)}{\frac{\alpha_{\min} T^r (C_1 + T^r)}{\alpha_{\min} T^r + C_1 + T^r}}$$

- $\alpha_{\max} T^r$:

$$\rho(\eta_1^{\text{ALG}}, C_1) = \frac{(1 - \eta_1^{\text{ALG}})\alpha_{\max} T^r}{\frac{\alpha_{\max} T^r (C_1 + T^r)}{\alpha_{\max} T^r + C_1 + T^r}}$$

Then,

$$\rho(C_1) = \min_{\eta_1^{\text{ALG}}} \max \left\{ \frac{\eta_1^{\text{ALG}}(C_1 + T^r)}{\frac{\alpha_{\min} T^r (C_1 + T^r)}{\alpha_{\min} T^r + C_1 + T^r}}, \frac{(1 - \eta_1^{\text{ALG}})\alpha_{\max} T^r}{\frac{\alpha_{\max} T^r (C_1 + T^r)}{\alpha_{\max} T^r + C_1 + T^r}} \right\}$$

Let $\beta = C_1/T^r$.

$$\eta_1^{\text{ALG}*} = \frac{\frac{\alpha_{\max} + \beta + 1}{\beta + 1}}{\frac{\alpha_{\min} + \beta + 1}{\beta + 1} + \frac{\alpha_{\max} + \beta + 1}{\beta + 1}}$$

Plugging $\eta_1^{\text{ALG}*}$ in ρ

$$\rho(\beta) = \frac{\frac{\alpha_{\max} + \beta + 1}{\beta + 1} \frac{\alpha_{\min} + \beta + 1}{\beta + 1}}{\frac{\alpha_{\max} + \beta + 1}{\beta + 1} + \frac{\alpha_{\min} + \beta + 1}{\beta + 1}}$$

$$\max_{\beta > 0} \rho(\beta) = \frac{1 + \sqrt{\frac{\alpha_{\max}}{\alpha_{\min}}}}{2}$$

APPENDIX C

STATIC RANDOMIZED ALGORITHM (DOPART-R)

Let μT^r be the optimal makespan for the given instance. Then $\mu = \min_i T_i/T^r$, where $\alpha_{\min} \leq \mu \leq \alpha_{\max}$. When the threshold $\hat{V}T^r < \mu T^r$ in line 9 of the algorithm, all the stages of the instance are processed locally for $T^l \leq \alpha_{\max} T^r$. When $\hat{V}T^r \geq \mu T^r$, the algorithm offloads for no more than $\hat{V}T^r$ makespan. Combining these two scenarios, the expected makespan of a thresholding algorithm that draws from a distribution $f_V(v)$ is given

$$\mathbb{E}[T_{\text{RAND}}] \leq \alpha_{\max} T^r \int_{\alpha_{\min}}^{\mu} f_V(v) dv + \int_{\mu}^{\alpha_{\max}} v T^r f_V(v) dv. \quad (30)$$

Then the algorithm is β competitive if

$$\mathbb{E}[T_{\text{RAND}}] \leq \beta T_{\text{OPT}} = \beta \mu T^r \quad \forall \mu \in (\alpha_{\min}, \alpha_{\max}). \quad (31)$$

Consider the following family of distributions

$$f_V^\epsilon(v) = \begin{cases} \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \frac{1}{\alpha_{\max} - v}, & v \in [\alpha_{\min}, (\alpha_{\max} - \epsilon)) \\ 1 + \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \ln \frac{\epsilon}{\alpha_{\max} - \alpha_{\min}}, & v = \alpha_{\max} \end{cases} \quad (32)$$

Recall that $T_{\text{OPT}} = \mu T^r$ where $T^r = \sum_{k=1}^L P_k^r$. On arrival of an instance I , the randomized algorithm draws $\hat{V} \in [\alpha_{\min}, (\alpha_{\max} - \epsilon)) \cup \alpha_{\max}$ with probability $f_V^\epsilon(v)$. It then sequentially processes the network locally and offloads when $T_i \leq \hat{V}T^r$.

Theorem 27. When the thresholds are drawn from $f_V^\epsilon(v)$, $\frac{\mathbb{E}[T_{\text{RAND}}]}{T_{\text{OPT}}} \leq \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon}$ for $\mu \in (\alpha_{\min}, \alpha_{\max})$.

Proof. When $\mu \in [\alpha_{\min}, (\alpha_{\max} - \epsilon))$,

$$\begin{aligned} \mathbb{E}[T_{\text{RAND}}] &\leq \alpha_{\max} T^r \int_{\alpha_{\min}}^{\mu} \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \frac{1}{\alpha_{\max} - v} dv \\ &\quad + \int_{\mu}^{\alpha_{\max} - \epsilon} v T^r \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \frac{1}{\alpha_{\max} - v} dv \\ &\quad + \alpha_{\max} T^r \left(1 + \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \ln \frac{\epsilon}{\alpha_{\max} - \alpha_{\min}} \right) \\ &= \alpha_{\max} T^r \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \ln \frac{\alpha_{\max} - \alpha_{\min}}{\alpha_{\max} - \mu} \\ &\quad + \alpha_{\max} T^r \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \ln \frac{\alpha_{\max} - \mu}{\epsilon} \\ &\quad + \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} T^r (\mu - (\alpha_{\max} - \epsilon)) \\ &\quad + \alpha_{\max} T^r \left(1 + \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \ln \frac{\epsilon}{\alpha_{\max} - \alpha_{\min}} \right) \\ &= \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \mu T^r \end{aligned} \quad (33)$$

Therefore,

$$\frac{\mathbb{E}[T_{\text{RAND}}]}{T_{\text{OPT}}} \leq \frac{\frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \mu T^r}{\mu T^r} = \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \quad (34)$$

When $\mu \in [(\alpha_{\max} - \epsilon), \alpha_{\max}]$,

$$\begin{aligned} \mathbb{E}[T_{\text{RAND}}] &\leq \alpha_{\max} T^r \int_{\alpha_{\min}}^{\alpha_{\max} - \epsilon} \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \frac{1}{\alpha_{\max} - v} dv \\ &\quad + \alpha_{\max} T^r \left(1 + \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \ln \frac{\epsilon}{\alpha_{\max} - \alpha_{\min}} \right) \\ &= \alpha_{\max} T^r \end{aligned} \quad (35)$$

Therefore,

$$\frac{\mathbb{E}[T_{\text{RAND}}]}{T_{\text{OPT}}} \leq \frac{\alpha_{\max} T^r}{(\alpha_{\max} - \epsilon) T^r} = \frac{\alpha_{\max}}{\alpha_{\max} - \epsilon} \quad (36)$$

□

Find an ϵ that minimizes the competitive ratio $\alpha_{\max}/(\alpha_{\max} - \epsilon)$ for a valid probability distribution $f_V^\epsilon(v)$.

$$\begin{aligned}
& \min_{\epsilon \in (0, \alpha^{\max}]} \frac{\alpha^{\max}}{\alpha^{\max} - \epsilon} \\
& \text{s.t. } 0 \leq 1 + \frac{\alpha^{\max}}{\alpha^{\max} - \epsilon} \ln \frac{\epsilon}{\alpha^{\max} - \alpha^{\min}} \leq 1
\end{aligned} \tag{37}$$

which is equivalent to,

$$\begin{aligned}
& \min_{\epsilon \in (0, \alpha^{\max} - \alpha^{\min}]} \frac{\alpha^{\max}}{\alpha^{\max} - \epsilon} \\
& \text{s.t. } 0 \leq 1 + \frac{\alpha^{\max}}{\alpha^{\max} - \epsilon} \ln \frac{\epsilon}{\alpha^{\max} - \alpha^{\min}}
\end{aligned} \tag{38}$$

Since $1 + \frac{\alpha^{\max}}{\alpha^{\max} - \epsilon} \ln \frac{\epsilon}{\alpha^{\max} - \alpha^{\min}}$ is increasing in ϵ and $\lim_{\epsilon \rightarrow 0} 1 + \frac{\alpha^{\max}}{\alpha^{\max} - \epsilon} \ln \frac{\epsilon}{\alpha^{\max} - \alpha^{\min}} < 0$, the solution to above optimization problem is obtained by solving following equality for ϵ

$$1 + \frac{\alpha^{\max}}{\alpha^{\max} - \epsilon} \ln \frac{\epsilon}{\alpha^{\max} - \alpha^{\min}} = 0 \tag{39}$$

The ϵ is given as $-\alpha^{\max} W\left(\frac{1 - \alpha^{\max}/\alpha^{\min}}{e\alpha^{\max}/\alpha^{\min}}\right)$, where $W(x)e^{W(x)} = x$.

APPENDIX D

RANDOMIZED LOWER-BOUND FIRST CASE

Define a *pruned* online delay minimization problem $\bar{P}$ over $\bar{\mathcal{I}} = \{I \in \mathcal{I} \mid P_i^l = P_i^r = 0, \forall i < L\}$. Then from the definition, $\bar{\mathcal{I}} \subset \mathcal{I}$. Similarly, define $\bar{P}'$ as the restriction of P' to $\bar{\mathcal{I}}$. Then, for any $\bar{I} \in \bar{\mathcal{I}}$, $T^{\min} = T_i^{\min} = \alpha_{\min} T^r$ and $T^{\max} = T_i^{\max} = \alpha_{\max} T^r$ for all $i < L$. Note that since $d_i \neq 0$, $C_i > 0$.

For a $\{s_k\}_{k=1}^{L+1}$ define

$$B_i = \frac{1}{L+1} \sum_{k=1}^{i-1} T_k + \left(s_i + \sum_{k=1}^{i-1} s_k - \frac{i-1}{L+1}\right) T_i + \sum_{k=i+1}^{L+1} s_k T_k. \tag{40}$$

Then we have the following lemma for consecutive B_i 's.

Lemma 28. *Given any $\{s_k\}_{k=1}^{L+1}$ if*

$$\sum_{k=1}^i s_k \geq \frac{i}{L+1}$$

for an i in $\{1, \dots, L\}$, then

$$B_i \geq B_{i+1}$$

and

$$\geq \frac{i}{L+1}$$

Proof. Let $B(s_1, \dots, s_{L+1}) = \sum_{k=1}^{L+1} s_k T_k$. $B(s_1, \dots, s_{L+1})$ is decreasing in s_{L+1} . Since $T_{i+1} \leq T_i \forall i \in \{1, \dots, L\}$, the minimum of $\sum_{k=1}^{L+1} s_k T_k$ over $\{s_k\}_{k=1}^{L+1}$ is achieved when $\hat{s}_i \leq \hat{s}_{i+1}$. From the definition of $\Delta^{L+1} \sum_{i=1}^{L+1} s_i = 1$. Then

$$1 - s_{L+1} = \sum_{i=1}^L s_i$$

$$\geq \frac{L}{L+1}$$

Then, $s_{L+1} \leq 1/(L+1)$. Then $B(s_1, \dots, s_{L+1}) \geq B(s_1, \dots, 1/(L+1))$ $\square$

$$V = \min_{\{s_k\}_{k=1}^{L+1}} \sum_{k=1}^{L+1} s_k T_k \tag{41}$$

$$\text{s.t. } \sum_{k=1}^i s_k \geq \frac{i}{L+1}, \quad \forall i \in \{1, \dots, L+1\}, \tag{42}$$

$$\sum_{k=1}^{L+1} s_k = 1, \tag{43}$$

$$s_k \geq 0, \quad \forall k \in \{1, \dots, L+1\}. \tag{44}$$

$$\min_{\{s_k\}_{k=1}^{L+1}} \left\{ \sum_{k=1}^{L+1} s_k T_k \mid \sum_{k=1}^i s_k \geq \frac{i}{L+1} \forall i, \sum_{k=1}^{L+1} s_k = 1, s_k \geq 0 \forall k \right\} \tag{45}$$

$$\geq \sum_{k=1}^{L+1} \frac{1}{L+1} T_k. \tag{46}$$

$$\begin{aligned}
V^* &:= \min_{\{s_k\}_{k=1}^{L+1}} \sum_{k=1}^{L+1} s_k T_k \\
&\text{s.t. } \sum_{k=1}^i s_k \geq \frac{i}{L+1}, \quad \forall i = 1, \dots, L+1, \\
&\sum_{k=1}^{L+1} s_k = 1, \\
&s_k \geq 0, \quad \forall k = 1, \dots, L+1.
\end{aligned} \tag{47}$$

$$V^* \geq \sum_{k=1}^{L+1} \frac{1}{L+1} T_k.$$

Proof. assume for The minimum of is obtained at $s_k = 1/(L+1) \forall k \in \{1, \dots, L+1\}$. Then

$$\sum_{k=1}^i s'_k = \frac{i}{L+1} \forall i \in \{1, \dots, L+1\}$$

and

$$\sum_{k=1}^{L+1} s'_k T_k = \sum_{k=1}^{L+1} \frac{1}{L+1} T_k. \tag{48}$$

For an arbitrary $v \in \{1, \dots, L+1\}$, $\sum_{k=1}^{L+1} s'_k T_k$ decreases in s'_v . When $s'_v < 1/(L+1)$ we have

$$\sum_{k=1}^{v-1} s'_k + \sum_{k=v+1}^{L+1} s'_k > \frac{L}{L+1}.$$

Then if

$$\sum_{k=v+1}^{L+1} s'_k > \frac{L-v+1}{L+1}. \quad (49)$$

$$\sum_{k=1}^{L+1} s'_k T_k > \sum_{k=1}^{L+1} \frac{1}{L+1} T_k$$

because

$$\sum_{k=1}^{v-1} s'_k > \frac{v-1}{L+1} \quad (50)$$

or

Since $\sum_k s'_k = 1$, s'_u increases $\square$

APPENDIX E GUESSING $\alpha_{\max}$

Proposition 29 (Impossibility of exactly learning $\alpha_{\max}$ online). *Suppose a sequence $\{\alpha_t\}_{t \geq 1}$ is revealed online and satisfies*

$$1 = \alpha_{\min} \leq \alpha_t \leq \alpha_{\max} \quad \forall t \geq 1,$$

where $\alpha_{\max}$ is unknown.

Then there is no online rule which, after observing only a finite prefix $\alpha_1, \dots, \alpha_t$, can always output the exact value of $\alpha_{\max}$ for every feasible sequence.

Proof. Fix any time t and any observed prefix $\alpha_1, \dots, \alpha_t$. Let

$$m_t := \max\{1, \alpha_1, \dots, \alpha_t\}.$$

For any number $B \geq m_t$, there exists a feasible continuation of the sequence whose true upper bound is exactly B .

Indeed, one continuation is to keep all future values at most m_t , in which case the true upper bound is m_t . Another continuation is to set $\alpha_{t+1} = B$ and keep all later values in $[1, B]$, in which case the true upper bound is B .

Hence the same observed prefix is compatible with multiple values of $\alpha_{\max}$. Therefore no online rule can identify the exact value of $\alpha_{\max}$ from a finite prefix in all cases. $\square$

Corollary 30 (Best possible online estimate without extra assumptions). *Under the same assumptions, the quantity*

$$\hat{\alpha}_t := \max\{1, \alpha_1, \dots, \alpha_t\}$$

is the largest universally valid lower estimate of $\alpha_{\max}$ that can be formed online from the observations up to time t . Moreover,

$$\hat{\alpha}_t \leq \alpha_{\max} \quad \text{for all } t,$$

and equality holds if and only if some observed value attains $\alpha_{\max}$.

Remark 31 (Adaptive guess of $\alpha_{\max}$). *Although $\alpha_{\max}$ cannot be learned exactly online in general, one may maintain a guess G_t and update it adaptively, for example by a doubling scheme:*

$$G_0 = 1, \quad G_{k+1} = 2G_k \quad \text{whenever an observation exceeds } G_k.$$

Such a scheme does not recover the exact value of $\alpha_{\max}$, but it ensures that after finitely many updates the maintained guess is within a constant factor of the true upper bound.

Proposition 32. *Knowing $\alpha_{\min} = 1$ only normalizes the scale; it does not make $\alpha_{\max}$ identifiable online. Without additional structural assumptions on the sequence $\{\alpha_t\}$, the exact value of $\alpha_{\max}$ cannot be determined from any finite observation history.*

APPENDIX F CVar $_{\delta}$ -COMPETITIVE ONE-MIN SEARCH: AN UPPER-BOUND CONSTRUCTION

We now consider the *one-min search* problem, the minimization analogue of one-max search. A player observes a sequence of prices $v_t \in [L, U]$ online, where $0 < L \leq U$, and wishes to *buy* one unit at the smallest possible price. The horizon T is unknown. After seeing price v_t , the player may either buy immediately at cost v_t , or wait for the next price. If the horizon ends before the player buys, a compulsory trade occurs at cost U . Let

$$\theta := \frac{U}{L}$$

be the fluctuation ratio.

As in the one-max setting, we restrict attention to randomized threshold algorithms: the algorithm draws a threshold X from a distribution supported on $[L, U]$, and buys at the first revealed price at most X . If the minimum price in the realized sequence is v , then the algorithm's realized cost is

$$C(X, v) := U \cdot \mathbf{1}_{\{X < v\}} + X \cdot \mathbf{1}_{\{X \geq v\}}.$$

Hence the CVar $_{\delta}$ -competitive ratio of a threshold distribution μ is

$$\alpha_{\delta}^{\text{OMin}(\theta), \mu} := \sup_{v \in [L, U]} \frac{\text{CVar}_{\delta}[U \cdot \mathbf{1}_{\{X < v\}} + X \cdot \mathbf{1}_{\{X \geq v\}}]}{v}.$$

We use the loss-form CVar:

$$\text{CVar}_{\delta}[Z] = \frac{1}{1-\delta} \int_{\delta}^1 F_Z^{-1}(p) dp.$$

Lemma 33. *Let X be a random variable supported on $[L, U]$, and fix an adversary choice of the minimal price $v \in [L, U]$. Let F_X denote the CDF of X . If X is continuous at v , then*

$$\text{CVar}_{\delta}[U \cdot \mathbf{1}_{\{X < v\}} + X \cdot \mathbf{1}_{\{X \geq v\}}] = \begin{cases} U, \\ \frac{1}{1-\delta} \left(F_X(v) U + \int_{F_X(v)+\delta}^1 F_X^{-1}(p) dp \right) \end{cases}$$

More generally, if X may have an atom at v , replace $F_X(v)$ by $F_X(v^-)$.

Proof. Fix $v \in [L, U]$, and write $C_v := U \cdot \mathbf{1}_{\{X < v\}} + X \cdot \mathbf{1}_{\{X \geq v\}}$. For $y < v$, clearly $F_{C_v}(y) = 0$. For $y \in [v, U]$,

$$F_{C_v}(y) = \Pr(C_v \leq y) = \Pr(v \leq X \leq y) = F_X(y) - F_X(v),$$

where continuity at v is used for the last equality. Finally, $F_{C_v}(U) = 1$. Thus

$$F_{C_v}^{-1}(p) = \begin{cases} F_X^{-1}(p + F_X(v)), & 0 \leq p < 1 - F_X(v), \\ U, & 1 - F_X(v) \leq p \leq 1. \end{cases}$$

Plugging this into

$$\text{CVaR}_\delta[C_v] = \frac{1}{1-\delta} \int_\delta^1 F_{C_v}^{-1}(p) dp$$

gives the result. If $F_X(v) \geq 1-\delta$, then the entire upper $(1-\delta)$ -tail consists of the cost U , so $\text{CVaR}_\delta[C_v] = U$. Otherwise,

$$\text{CVaR}_\delta[C_v] = \frac{1}{1-\delta} \left(\int_\delta^{1-F_X(v)} F_X^{-1}(p + F_X(v)) dp + \int_{1-F_X(v)}^1 U dp \right) = \frac{1}{1-\delta} \left(yU + \int_{y+\delta}^1 \phi(t) dt \right), \quad y \in [0, 1-\delta].$$

and the change of variables $t = p + F_X(v)$ yields the claimed expression. $\square$

The preceding lemma suggests an equalizer-style construction analogous to the one-max case.

Theorem 34. Fix $\delta \in [0, 1]$. Let $\psi : [0, 1] \rightarrow [L, U]$ satisfy

$$\psi'(t) = \frac{\psi(t-\delta) - U}{\alpha_\delta(1-\delta)}, \quad t \in [\delta, 1],$$

with initial condition

$$\psi(t) = \frac{U}{\alpha_\delta}, \quad t \in [0, \delta],$$

where α_δ is chosen so that $\psi(1) = L$ when $\delta < 1$, and $\alpha_1 := \sqrt{\theta}$. Define

$$\phi(y) := \psi(1-y), \quad y \in [0, 1].$$

Then ϕ is the inverse CDF of a random threshold algorithm for one-min search whose CVaR $_\delta$ -competitive ratio is at most α_δ .

Equivalently, for $\delta < 1$, ϕ satisfies

$$\alpha_\delta(1-\delta)\phi'(y) = U - \phi(y+\delta), \quad y \in [0, 1-\delta],$$

with terminal condition

$$\phi(y) = \frac{U}{\alpha_\delta}, \quad y \in [1-\delta, 1].$$

Proof. We first consider $\delta < 1$. Since $\psi(t) \leq U$, the differential equation implies $\psi'(t) \leq 0$ on $[\delta, 1]$; therefore ψ is nonincreasing, and so $\phi(y) = \psi(1-y)$ is nondecreasing. Also,

$$\phi(0) = \psi(1) = L, \quad \phi(1) = \psi(0) = \frac{U}{\alpha_\delta} \leq U,$$

so ϕ is the inverse CDF of a threshold distribution supported on $[L, U/\alpha_\delta]$.

Now let X be the threshold random variable with inverse CDF ϕ . For $y \in [1-\delta, 1]$, we have $\phi(y) = U/\alpha_\delta$, hence

$$\alpha_\delta\phi(y) = U.$$

Thus for any v with $F_X(v) = y \geq 1-\delta$, Lemma 33 gives

$$\text{CVaR}_\delta[C(X, v)] = U = \alpha_\delta v.$$

Next, fix $y \in [0, 1-\delta]$, and define

$$H(y) := \alpha_\delta(1-\delta)\phi(y) - yU - \int_{y+\delta}^1 \phi(t) dt.$$

Differentiating and using the advanced form of the differential equation gives

$$H'(y) = \alpha_\delta(1-\delta)\phi'(y) - U + \phi(y+\delta) = 0.$$

So H is constant on $[0, 1-\delta]$. Evaluating at $y = 1-\delta$,

$$H(1-\delta) = \alpha_\delta(1-\delta)\phi(1-\delta) - (1-\delta)U = (1-\delta)(\alpha_\delta\phi(1-\delta) - U) = 0,$$

because $\phi(1-\delta) = U/\alpha_\delta$. Hence $H(y) \equiv 0$, i.e.,

$$\alpha_\delta\phi(y) = yU + \int_{y+\delta}^1 \phi(t) dt, \quad y \in [0, 1-\delta].$$

If v is such that $F_X(v) = y < 1-\delta$, Lemma 33 yields exactly

$$\text{CVaR}_\delta[C(X, v)] = \alpha_\delta v.$$

Therefore,

$$\text{CVaR}_\delta[C(X, v)] \leq \alpha_\delta v \quad \text{for all } v \in [L, U],$$

and so the resulting algorithm is α_δ -competitive in the CVaR $_\delta$ sense.

Finally, when $\delta = 1$, CVaR reduces to the essential supremum. The deterministic threshold $\sqrt{LU}$ has worst-case ratio

$$\max \left\{ \frac{\sqrt{LU}}{L}, \frac{U}{\sqrt{LU}} \right\} = \sqrt{\theta},$$

so $\alpha_1 = \sqrt{\theta}$. $\square$

As in the one-max case, the delay differential equation can be solved explicitly by integrating step-by-step. Let

$$z(t) := U - \psi(t).$$

Then $z(t) = U \left(1 - \frac{1}{\alpha_\delta}\right)$ on $[0, \delta]$, and

$$z'(t) = \frac{z(t-\delta)}{\alpha_\delta(1-\delta)}, \quad t \in [\delta, 1].$$

Hence

$$z(t) = U \left(1 - \frac{1}{\alpha_\delta}\right) \sum_{j=0}^{\infty} \frac{([t-j\delta]_+)^j}{(\alpha_\delta(1-\delta))^j j!},$$

and therefore

$$\psi(t) = U - U \left(1 - \frac{1}{\alpha_\delta}\right) \sum_{j=0}^{\infty} \frac{([t-j\delta]_+)^j}{(\alpha_\delta(1-\delta))^j j!}.$$

Imposing $\psi(1) = L$ yields the implicit equation

$$\left(1 - \frac{1}{\alpha_\delta}\right) \sum_{j=0}^{\infty} \frac{([1-j\delta]_+)^j}{(\alpha_\delta(1-\delta))^j j!} = 1 - \frac{1}{\theta}.$$

When $\delta > 0$, only finitely many terms are nonzero.

Two useful special cases are:

$$\delta = 0 : \quad \left(1 - \frac{1}{\alpha_0}\right) e^{1/\alpha_0} = 1 - \frac{1}{\theta},$$

$$\delta \geq \frac{1}{2} : \quad \psi(1) = U - U \left(1 - \frac{1}{\alpha_\delta}\right) \left(1 + \frac{1}{\alpha_\delta}\right) = \frac{U}{\alpha_\delta^2},$$

so that $\psi(1) = L$ implies

$$\alpha_\delta = \sqrt{\theta}, \quad \delta \geq \frac{1}{2}.$$

Thus, this upper-bound construction exhibits the same threshold $\delta = \frac{1}{2}$ at which the achieved CVaR $_\delta$ -competitive ratio coincides with the deterministic value $\sqrt{\theta}$. A matching lower bound, however, requires a separate argument and is not established here.

REFERENCES

- [1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [2] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra, and P. Bahl, "Maui: making smartphones last longer with code offload," in *Proc. International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2010, pp. 49–62.
- [3] J. P. Champati and B. Liang, "Semi-online algorithms for computational task offloading with communication delay," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 4, pp. 1189–1201, 2016.
- [4] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 615–629, 2017.
- [5] Y. Huang, F. Wang, F. Wang, and J. Liu, "Deepar: A hybrid device-edge-cloud execution framework for mobile deep learning applications," in *Proc. IEEE INFOCOM WKSHPS*, 2019, pp. 892–897.
- [6] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach," *IEEE Transactions on Wireless Communications*, vol. 19, no. 8, pp. 5404–5419, 2020.
- [7] J. Wang, J. Hu, G. Min, W. Zhan, A. Y. Zomaya, and N. Georgalas, "Dependent task offloading for edge computing based on deep reinforcement learning," *IEEE Trans. Comp.*, vol. 71, no. 10, pp. 2449–2461, 2021.
- [8] L. Zhang, L. Chen, and J. Xu, "Autodidactic neurosurgeon: Collaborative deep inference for mobile edge intelligence via online learning," in *Proc. the Web Conference 2021*, 2021, pp. 3111–3123.
- [9] H. Li, K. Ota, and M. Dong, "Learning iot in edge: Deep learning for the internet of things with edge computing," *IEEE Network*, vol. 32, no. 1, pp. 96–101, 2018.
- [10] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive dnn surgery for inference acceleration on the edge," in *Proc. IEEE Conference on Computer Communications (INFOCOM)*, 2019, pp. 1423–1431.
- [11] S. Zhang, Y. Li, X. Liu, S. Guo, W. Wang, J. Wang, B. Ding, and D. Wu, "Towards real-time cooperative deep inference over the cloud and edge end devices," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 4, no. 2, pp. 1–24, 2020.
- [12] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "Jointdnn: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Transactions on Mobile Computing*, vol. 20, no. 2, pp. 565–576, 2019.
- [13] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 328–339.
- [14] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447–457, 2019.
- [15] H. Li, C. Hu, J. Jiang, Z. Wang, Y. Wen, and W. Zhu, "Jalad: Joint accuracy-and latency-aware deep structure decoupling for edge-cloud execution," in *Proc. IEEE ICPADS*. IEEE, 2018, pp. 671–678.
- [16] L. Wang, L. Xiang, J. Xu, J. Chen, X. Zhao, D. Yao, X. Wang, and B. Li, "Context-aware deep model compression for edge cloud computing," in *Proc. IEEE ICDCS*. IEEE, 2020, pp. 787–797.
- [17] L. Yang, J. Cao, Y. Yuan, T. Li, A. Han, and A. Chan, "A framework for partitioning and execution of data stream applications in mobile cloud computing," *ACM SIGMETRICS Performance Evaluation Review*, vol. 40, no. 4, pp. 23–32, 2013.
- [18] M. Jia, J. Cao, and L. Yang, "Heuristic offloading of concurrent tasks for computation-intensive applications in mobile cloud computing," in *Proc. IEEE INFOCOM WKSHPS*, 2014, pp. 352–357.
- [19] H. Wu, W. J. Knottenbelt, and K. Wolter, "An efficient application partitioning algorithm in mobile environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 7, pp. 1464–1480, 2019.
- [20] L. Yang, J. Cao, H. Cheng, and Y. Ji, "Multi-user computation partitioning for latency sensitive mobile cloud applications," *IEEE Transactions on Computers*, vol. 64, no. 8, pp. 2253–2266, 2014.
- [21] S. Liu, Y. Yu, X. Lian, Y. Feng, C. She, P. L. Yeoh, L. Guo, B. Vucetic, and Y. Li, "Dependent task scheduling and offloading for minimizing deadline violation ratio in mobile edge computing networks," *IEEE J. Selected Areas in Commun.*, vol. 41, no. 2, pp. 538–554, 2023.
- [22] F. Wang, S. Cai, and V. K. Lau, "Sequential offloading for distributed DNN computation in multiuser MEC systems," *IEEE Internet of Things Journal*, vol. 10, no. 20, pp. 18 315–18 329, 2023.
- [23] R. El-Yaniv, A. Fiat, R. M. Karp, and G. Turpin, "Optimal search and one-way trading online algorithms," *Algorithmica*, vol. 30, pp. 101–139, 2001.